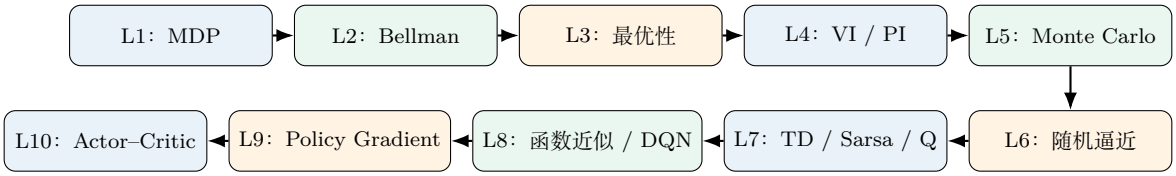
强化学习 L1–L10 核心归纳

从 MDP、价值学习到策略梯度与 Actor–Critic



十讲形成的完整闭环

MDP 建模 → Bellman 方程 → 模型/样本求价值 → 函数近似
→ 直接优化策略 → Actor–Critic

L1–L8 建立 value-based 主线；L9 建立 policy-based 主线；L10 用 TD critic 与策略梯度 actor 把两条主线合在同一个学习循环中。

初学者复习版：统一符号、核心公式、直观解释、算法比较、易错点与自测

依据工作区 `Book` 目录 Chapter 1–10、对应 slides 与勘误表核对

目录

统一记号与阅读方法 **4**

最常用记号；读 Bellman 公式的固定顺序 4

L1：Basic Concepts——MDP 的对象与回报 **5**

状态、动作、奖励、转移、策略、回报与 Markov 性 5–7

L2：State Values and Bellman Equation **8**

状态价值、Bellman 期望方程、矩阵形式与动作价值 8–10

L3：Optimal Values and Bellman Optimality **11**

最优价值、Bellman 最优方程、压缩映射与策略提取 11–13

L4：Value Iteration and Policy Iteration **14**

价值迭代、策略迭代、截断策略迭代及比较 14–16

L5：Monte Carlo Methods **17**

完整回报、Basic MC、visit 类型、探索与控制 17–20

前五讲的定位

L1–L4 从 MDP 和 Bellman 方程出发，在模型已知时进行动态规划；L5 第一次用真实轨迹回报代替模型期望，进入 model-free 学习。

目录（续）

L6：Stochastic Approximation **21**

增量均值、Robbins–Monro、步长条件与 SGD 21–24

L7：Temporal-Difference Methods **25**

TD、Sarsa、Expected/n-step Sarsa 与 Q-learning 25–33

L8：Value Function Methods **34**

函数近似、TD-Linear、投影 Bellman 方程与 DQN 34–41

L9：Policy Gradient Methods **42**

策略参数化、优化指标与策略梯度定理 42

log-derivative trick 与 REINFORCE 42

L10：Actor–Critic Methods **47**

QAC、baseline、advantage 与 A2C 47

Off-policy AC、importance sampling 与 DPG 47

L1–L10 跨章节串联 **53**

内容核对与专业性说明 **55**

统一记号与阅读方法

最常用的记号

记号	名称	一句话解释
S_t, A_t, R_{t+1}	状态、动作、奖励	时刻 t 看见 S_t，选择 A_t，环境随后给出 R_{t+1} 和 S_{t+1}。
$\pi(a \mid s)$	策略	在状态 s 选择动作 a 的概率。
$p(s', r \mid s, a)$	环境模型	执行 a 后同时得到下一状态 s' 和奖励 r 的概率。
γ	折扣因子	控制未来奖励在当前看来有多重要，通常 $0 \le \gamma < 1$。
G_t	回报	从时刻 t 往后收到的折扣奖励总和。
$v_\pi(s), q_\pi(s, a)$	策略 π 的价值	分别评价“从状态开始”和“从状态–动作对开始”有多好。
$v_*(s), q_*(s, a)$	最优价值	所有策略能达到的最好价值。

读公式的固定顺序

看到任何 Bellman 公式时，不要先背求和符号。按下面三步读：

1. **当前先拿到什么？**——即时奖励 R_{t+1}；
2. **之后还剩什么？**——从下一状态开始的价值；
3. **谁决定动作？**——固定策略时由 π 加权，求最优时对动作取最大值。

全书最重要的一句话

$$\boxed{\text{当前价值} = \text{即时奖励} + \gamma \times \text{下一状态的价值}}$$

L2、L3、L4 的各种公式，都是这句话在不同问题下的具体写法。

1　L1：基本概念——把交互问题写成 MDP

本章要回答的问题

智能体面对环境时，什么是状态、动作、策略、奖励和回报？怎样用一个统一的数学对象描述这些元素？

1.1　智能体与环境如何交互

强化学习不是一次性预测，而是一个连续反馈过程：

$$S_t \xrightarrow[\text{智能体按策略选择}]{A_t} (R_{t+1}, S_{t+1}) \xrightarrow{\text{继续决策}} \cdots$$

智能体看见当前状态 S_t，选择动作 A_t；环境响应这个动作，给出奖励 R_{t+1} 和下一状态 S_{t+1}。智能体的目标不是让某一步的奖励最大，而是让**长期累计回报**尽可能大。

1.2　五个基础概念

1.2.1　状态：对决策有用的环境信息

状态 $s \in \mathcal{S}$ 是智能体用来决策的环境描述。在网格世界中，智能体所在的格子可以是状态；在机器人任务中，位置、速度、传感器读数都可能属于状态。

易错点

“状态”不只是随便记录一些观测。理想的状态应包含预测未来所需的信息。如果同一份当前描述对应着不同的未来规律，说明它可能遗漏了历史信息，此时直接把它当作 MDP 状态会有问题。

1.2.2　动作：智能体可以作出的选择

动作集合记作 $\mathcal{A}(s)$。不同状态允许的动作可以不同。例如边界处不能继续向外走；也可以把“向外走”保留为动作，但规定智能体留在原处并受到惩罚。两种建模方式都可以，关键是把规则写清楚。

1.2.3　策略：从状态到动作分布的决策规则

策略写成

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s).$$

如果每个状态只选择一个确定动作，它是**确定性策略**；如果在多个动作之间按概率选择，它是**随机策略**。对每个状态都必须满足

$$\pi(a \mid s) \geq 0, \qquad \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) = 1.$$

1.2.4　环境模型：动作会把世界变成什么样

最一般的单步模型可写为

$$p(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

它同时描述下一状态和奖励的不确定性。确定性网格世界只是特殊情况：给定 s, a 后，某个 s', r 的概率为 1，其余为 0。

课件也分别使用状态转移概率 $p(s' \mid s, a)$ 与奖励概率 $p(r \mid s, a)$。计算期望时可以使用这两个边缘分布；联合形式 $p(s', r \mid s, a)$ 则能保留二者之间可能存在的相关性。

1.2.5　奖励：环境给出的即时反馈

奖励是一步交互后的标量反馈。它告诉智能体“发生了什么结果”，而不直接告诉智能体“正确动作是哪一个”。奖励设计实际上定义了任务目标：

- 到达目标给正奖励；
- 撞墙或进入危险区域给负奖励；
- 普通移动给零奖励或很小的步长代价。

奖励不是价值

奖励 R_{t+1} 只评价刚完成的一步；价值评价从现在起的整个未来。一个动作眼前奖励较小，却可能通向长期回报更高的状态。

1.3　轨迹、回合与回报

1.3.1　轨迹与回合

一条轨迹是交互序列

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots$$

如果任务在终止状态结束，这条有限轨迹称为一个**回合（episode）**，任务称为回合式任务；如果交互没有自然终点，则称为持续式任务。

终止状态之后通常不再产生奖励，可等价地令终止状态的后续价值为 0；也可以增加一个零奖励的吸收状态。两种写法表达的是同一件事。

1.3.2　为什么要有折扣回报

若只把所有奖励直接相加，无穷轨迹的和可能发散。与此同时，我们通常也希望越晚得到的奖励对现在的影响越弱。因此定义

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

这里第一个奖励没有折扣，晚一步的奖励多乘一个 γ：

- γ 小：更看重眼前，策略更“短视”；
- γ 接近 1：更重视长期结果，但价值传播和算法收敛通常更慢；
- $\gamma < 1$ 且奖励有界时，无穷和是有限的。

1.4　Markov 性与 MDP

先抓直觉

如果已经知道当前状态和当前动作，那么预测下一步时不应再需要完整历史。也就是说，当前状态应当是过去信息对未来的充分摘要。

这一点称为 Markov 性：

$$\mathbb{P}(S_{t+1}, R_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \ldots) = \mathbb{P}(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

若把状态转移与奖励分别描述，一个 MDP 通常写成五元组

$$\boxed{(\mathcal{S}, \mathcal{A}, P, R, \gamma)}.$$

其中 $\mathcal{S}$ 是状态集合，$\mathcal{A}$ 是动作集合，P 是状态转移规律，R 是奖励规律，γ 是折扣因子。也可以用前面的联合分布 $p(s', r \mid s, a)$ 把 P 与 R 合在一起描述。策略 π 不是环境本身的一部分，而是智能体要选择或学习的对象。

关于“奖励整体平移”的适用条件

在无限时域折扣任务中，把所有奖励变成 $ar + b$（其中 $a > 0$）不会改变最优策略，因为每条无限轨迹都增加同样的 $b/(1-\gamma)$，且正比例缩放不改变大小关系。但在长度不同的回合式任务中，每走一步都加 b 会偏好更长或更短的回合，因此不能不加条件地说“奖励平移永远不改变策略”。

本章速记

- MDP 描述环境，策略描述智能体怎么行动。
- 奖励是一时的反馈，回报是未来奖励之和。
- 折扣因子同时影响远期偏好、回报是否有限以及算法收敛速度。
- Markov 性要求当前状态足以预测下一步。
- 到 L2 后，真正被计算和比较的是**期望回报，也就是价值**。

2　L2：状态价值与 Bellman 期望方程

本章要回答的问题

给定一个策略 π，怎样评价它在每个状态有多好？为什么价值之间会互相依赖，又怎样把它们计算出来？

2.1　从一次回报到平均表现

随机策略和随机环境会让同一个起点产生不同轨迹，因此一次实际回报 G_t 不能稳定代表策略质量。更合理的评价是：从该状态出发，把所有可能未来按发生概率取平均。

2.1.1　状态价值

$$\boxed{v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]}$$

下标 π 提醒我们：改变策略后，即使环境没变，状态价值也可能改变。状态价值回答的是：“身处 s，以后一直按 π 行动，平均能得到多少回报？”

2.1.2　动作价值

$$\boxed{q_\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]}$$

它回答：“在 s 先强制做动作 a，以后再按 π 行动，平均能得到多少回报？”正因为它分别评价每个动作，所以后面可以用它改进策略。

状态价值与动作价值的关系

在状态 s，策略会以概率 $\pi(a \mid s)$ 选择各个动作。因此，状态价值就是这些动作价值的加权平均：

$$\boxed{v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a \mid s) q_\pi(s,a)}.$$

2.2　Bellman 方程为什么会出现

2.2.1　第一步：把回报拆成“现在 + 未来”

从回报定义出发：

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \\ &= R_{t+1} + \gamma\big(R_{t+2} + \gamma R_{t+3} + \cdots\big) \\ &= \boxed{R_{t+1} + \gamma G_{t+1}}. \end{aligned}$$

最后一行不是额外假设，只是把同一个无穷和重新分组。

2.2.2　第二步：在固定策略下取条件期望

站在状态 s：

1. 策略以 $\pi(a \mid s)$ 的概率选择动作 a；
2. 环境以 $p(s', r \mid s, a)$ 的概率给出奖励 r 和下一状态 s'；
3. 从 s' 开始，剩余期望回报就是 $v_\pi(s')$。

把所有可能的 a, s', r 按概率加权，得到

$$\boxed{v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s') \right]}.$$

这就是 **Bellman 期望方程**。它不是一个额外的学习规则，而是价值定义在 Markov 性下必然满足的自洽关系。

2.3　动作价值的 Bellman 形式

若第一步动作 a 已经给定，就不再需要对当前动作按 π 求平均：

$$\boxed{q_\pi(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s') \right]}.$$

再代入 $v_\pi(s') = \sum_{a'} \pi(a' \mid s') q_\pi(s', a')$，也可以得到只含 q_π 的递推式。记忆时不必死背展开形式，只需记住：**第一步固定为 a，下一步起重新服从 π**。

2.4　一个两状态例子

考虑固定策略下的确定性过程：

- 从 s_1 得到奖励 0，然后进入 s_2；
- 在 s_2 每一步得到奖励 1，并留在 s_2。

先从容易的 s_2 看。它现在得到 1，下一步又回到同一个状态，所以

$$v_\pi(s_2) = 1 + \gamma v_\pi(s_2).$$

移项后得到

$$v_\pi(s_2) = \frac{1}{1 - \gamma}.$$

而 s_1 当前奖励为 0，随后进入 s_2，因此

$$v_\pi(s_1) = 0 + \gamma v_\pi(s_2) = \frac{\gamma}{1 - \gamma}.$$

这个例子把 Bellman 方程的含义展示得很清楚：**一个状态的价值，可以借用下一状态的价值来表达**。这种“用已有估计更新另一个估计”的思想称为 **bootstrapping**。

2.5　怎样求出所有状态价值

2.5.1　矩阵形式

对固定策略，把每个状态的平均即时奖励放进向量 $\boldsymbol{r}_\pi$，把策略诱导出的状态转移概率放进矩阵 P_π，所有状态价值放进 $\boldsymbol{v}_\pi$。各状态的 Bellman 方程可一起写成

$$\boxed{\boldsymbol{v}_\pi = \boldsymbol{r}_\pi + \gamma P_\pi \boldsymbol{v}_\pi}.$$

如果状态数有限且 $\gamma < 1$，则 $I - \gamma P_\pi$ 可逆，于是

$$\boldsymbol{v}_\pi = (I - \gamma P_\pi)^{-1} \boldsymbol{r}_\pi.$$

这是理论上的闭式表达，但大规模问题通常不显式求逆。

2.5.2　迭代策略评价

Bellman 方程本身已经暗示了一种迭代方法。先任取初值 $\boldsymbol{v}_0$，然后反复执行

$$\boxed{\boldsymbol{v}_{k+1} = \boldsymbol{r}_\pi + \gamma P_\pi \boldsymbol{v}_k}.$$

每轮都用旧价值计算“一步奖励 + 折扣后的下一状态价值”。在本文假设下，无论初值如何，这个序列都会收敛到真正的 $\boldsymbol{v}_\pi$。

下标最容易混淆的地方

v_π 中的 π 表示“这是谁的价值”；v_k 中的 k 表示“当前是第几次近似”。迭代尚未收敛时，v_k 一般还不等于任何策略的精确价值。

本章速记

- 固定策略评价的对象是 v_π 和 q_π，二者都是期望回报。
- Bellman 期望方程来自 $G_t = R_{t+1} + \gamma G_{t+1}$。
- 固定策略时，对动作按 $\pi(a \mid s)$ 加权，而不是取最大。
- v_π 是当前策略各动作价值的加权平均。
- 求出 v_π 叫作**策略评价**；它为下一步策略改进提供依据。

3　L3：最优策略与 Bellman 最优方程

本章要回答的问题

怎样严格定义“更好的策略”？最优价值满足什么方程？为什么解这个方程就能得到最优策略？

3.1　从评价转向控制

L2 的问题是：**给定策略，算它有多好**。 L3 的问题变为：**在所有策略中，找一个最好的**。

若对所有状态都有

$$v_{\pi_1}(s) \geq v_{\pi_2}(s),$$

就说 π_1 不劣于 π_2。最优状态价值定义为

$$\boxed{v_*(s) = \max_{\pi} v_\pi(s)}.$$

最优动作价值定义为

$$\boxed{q_*(s,a) = \max_{\pi} q_\pi(s,a)}.$$

在有限折扣 MDP 中，至少存在一个平稳、确定性的最优策略。最优策略不一定唯一：如果多个动作并列最优，它们都可以被选；只在这些并列最优动作上随机化，也仍然是最优策略。

3.2　Bellman 最优方程怎样得到

先抓直觉

站在状态 s，最优决策可以分成两部分：**当前先选一个最好的动作**，然后从下一状态开始继续最优地行动。因此“固定策略下的平均”会被“对动作取最大”替代。

若当前选择动作 a，一步之后的最佳期望回报为

$$\sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right].$$

这就是最优动作价值：

$$\boxed{q_*(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]}.$$

当前状态应该选价值最大的动作，于是

$$\boxed{v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s,a)}.$$

把两式合并，得到 **Bellman 最优方程**：

$$\boxed{v_*(s) = \max_{a \in \mathcal{A}(s)} \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right]}.$$

3.3　期望方程与最优方程不要混

	Bellman 期望方程	Bellman 最优方程
回答的问题	给定 π，它的价值是多少?	所有策略中最好的价值是多少?
动作如何处理	按 $\pi(a \mid s)$ 加权平均	对 a 取最大值
未知量	v_π 或 q_π	v_* 或 q_*
方程性质	固定 π 后是线性的	含有 max，通常是非线性的
对应任务	策略评价	最优控制

3.4　为什么最优方程有唯一解

定义 Bellman 最优算子

$$(\mathcal{T}_* v)(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v(s') \right].$$

Bellman 最优方程就是固定点方程

$$v = \mathcal{T}_* v.$$

“压缩”是什么意思？

假设两个价值猜测 v, w 最多相差 d。经过一次 Bellman 更新后，即时奖励部分相同，差异只来自下一状态价值，而这一部分还要乘 γ。因此新的最大差异至多是 γd。

用无穷范数 $\|v\|_\infty = \max_s |v(s)|$ 表示“所有状态中的最大误差”，可以严格写成

$$\boxed{\|\mathcal{T}_* v - \mathcal{T}_* w\|_\infty \le \gamma \|v - w\|_\infty}.$$

因为 $\gamma < 1$，每次更新都会把最坏误差按比例缩小。压缩映射定理因此给出三个重要结论：

1. **存在性：** Bellman 最优方程至少有一个解；
2. **唯一性：** 这个解只有一个，也就是 v_*；
3. **可计算性：** 从任意 v_0 开始反复做 $v_{k+1} = \mathcal{T}_* v_k$，都会收敛到 v_*。

范数条件

Bellman 最优算子标准地是在**无穷范数**下具有折扣因子 γ 的压缩性。不能把它无条件扩展为“在任意向量范数下都是 γ-压缩”。

3.5　从最优价值取出最优策略

一旦知道 v_*，就在每个状态计算“一步奖励 + 最优后续价值”，并选择最大的动作：

$$a_*(s) \in \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right].$$

令

$$\pi_*(a \mid s) = \begin{cases} 1, & a = a_*(s), \\ 0, & a \neq a_*(s), \end{cases}$$

便得到一个确定性最优策略。这叫对 v_* **贪心**。

“贪心”并不等于只看即时奖励

这里比较的是 $r + \gamma v_*(s')$，它已经包含未来的最优价值；所以这是对长期回报贪心，而不是只挑眼前奖励最大的动作。

3.6　什么会改变最优策略

- **环境模型：**相同动作若通向不同状态，最优选择当然会变；
- **奖励设计：**更强的危险惩罚会让策略更保守；
- **折扣因子：**γ 越小越重眼前，可能不愿为远期目标绕路；
- **正比例缩放：**在通常条件下，$r \mapsto ar$、$a > 0$ 不改变大小关系，因此不改变最优策略；
- **常数平移：**对无限时域折扣任务不改变最优策略，但对长度可变的回合式任务需要谨慎。

本章速记

- 固定策略：对动作求平均；追求最优：对动作取最大。
- $v_*(s) = \max_a q_*(s, a)$。
- Bellman 最优算子在无穷范数下是 γ-压缩映射。
- 因而最优价值存在、唯一，而且可以从任意初值迭代求出。
- 对最优价值做一步贪心，就能得到一个最优策略。

4　L4：价值迭代、策略迭代与截断策略迭代

本章要回答的问题

L3 已经知道最优价值满足什么方程。现在怎样真正把它算出来？价值迭代和策略迭代有何区别，又有什么共同结构？

4.1　先明确：这是有模型的动态规划

L4 的算法需要知道 $p(s', r \mid s, a)$，因此属于**动态规划（DP）**和**基于模型**的方法。后面的 Monte Carlo 与 TD 方法会讨论模型未知时，怎样利用采样到的经验代替模型中的期望。

4.2　价值迭代（Value Iteration, VI）

4.2.1　算法从哪里来

L3 已知 Bellman 最优方程是固定点方程 $v_* = \mathcal{T}_* v_*$，而压缩映射定理告诉我们可以从任意 v_0 反复应用 $\mathcal{T}_*$。因此

$$\boxed{v_{k+1}(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_k(s') \right]}.$$

为了更容易读，可以把一轮拆成两步。先对每个动作做一步展望：

$$q_k(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_k(s') \right],$$

再选最大者：

$$v_{k+1}(s) = \max_a q_k(s, a).$$

4.2.2　初学者版伪代码

价值迭代的一轮

1. 任意初始化每个状态的数值 $v_0(s)$；
2. 对每个状态 s，用旧表 v_k 计算每个动作的 $q_k(s, a)$；
3. 把最大动作价值写入新表：$v_{k+1}(s) = \max_a q_k(s, a)$；
4. 重复，直到相邻两轮的最大变化足够小；
5. 最后对收敛后的价值表做贪心，取出策略。

易错点

中间的 v_k 只是对 v_* 的近似，一般不必是某个策略的精确状态价值。价值迭代的核心是直接逼近 Bellman 最优方程的固定点。

4.3　策略迭代（Policy Iteration, PI）

策略迭代显式维护一个策略，并反复进行**策略评价**与**策略改进**。

4.3.1　第一步：策略评价

给定当前策略 π_k，求它的精确价值：

$$\boxed{v_{\pi_k}(s) = \sum_a \pi_k(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_{\pi_k}(s') \right]}.$$

这就是 L2 的 Bellman 期望方程。可通过解线性方程组，或反复应用 $\mathcal{T}_\pi$ 直到收敛来完成。

4.3.2　第二步：策略改进

利用刚算出的 v_{π_k}，对每个状态做一步展望：

$$q_{\pi_k}(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_{\pi_k}(s') \right].$$

然后选取动作价值最大的动作

$$\boxed{a_{k+1}(s) \in \arg\max_a q_{\pi_k}(s, a)},$$

并令新策略在状态 s 确定地选择 $a_{k+1}(s)$。

为什么改进后不会更差？

$v_{\pi_k}(s)$ 是按旧策略选择动作所得 $q_{\pi_k}(s, a)$ 的平均值。最大值不会小于平均值，所以在每个状态改选最大动作，至少不会让“一步之后继续按旧策略”的回报变差。策略改进定理进一步保证

$$v_{\pi_{k+1}}(s) \geq v_{\pi_k}(s), \qquad \forall s.$$

4.3.3　何时停止

如果贪心改进后策略不再变化，当前策略满足对自身价值贪心，它就是最优策略。在有限 MDP 中，配合一致的并列动作处理方式，策略迭代会在有限次策略改进后停在某个最优策略。

并列动作

多个动作价值相同时，若实现中每轮随意切换并列动作，可能出现“策略看起来一直在变、价值却没有变”的现象。实际编程时应使用固定的 tie-breaking 规则，或把“只在并列动作间变化”也视为已经稳定。

4.4　截断策略迭代（Modified/Truncated PI）

精确策略评价可能很贵。一个自然的折中是：每次只做有限轮策略评价，就提前进行策略改进。这通常称为**截断策略迭代**或**修正策略迭代**。

设每次策略评价只执行 m 次：

- $m = 1$：只做一次 Bellman 更新，行为接近价值迭代；
- m 较大：更充分地评价当前策略；
- $m \to \infty$：得到精确策略评价，对应标准策略迭代。

因此 VI 与 PI 不是毫无关系的两套算法，而是“评价深度”这条轴线上的两个端点。

4.5　三种算法的对比

	价值迭代	截断策略迭代	策略迭代
维护的核心对象	价值近似 v_k	策略与近似评价值	策略与精确评价值
每次评价深度	一步	固定或自适应的有限步	直到评价收敛或精确求解
单轮开销	通常较小	居中	通常较大
外层轮数	可能较多	居中	常较少
中间价值是否对应某策略	一般不对应精确价值	近似对应当前策略	是当前策略的精确价值
共同前提	知道环境模型；都利用 Bellman 更新与贪心改进。		

4.6　实现时最值得记住的四点

1. **同步更新与原地更新要分清**。公式通常用完整的旧表 v_k 生成新表 v_{k+1}；原地更新会立刻使用本轮刚得到的新值，通常也能工作，但更新顺序会影响过程。
2. **用最大变化监测收敛**。常见指标是 $\Delta_k = \max_s |v_{k+1}(s) - v_k(s)|$。它足够小时停止，但阈值越小，计算越久。
3. **γ 越接近 1，通常收敛越慢**。因为压缩比例就是 γ，误差每轮缩小得更少。
4. **模型未知时不能直接照搬**。求和中的转移概率和奖励规律未知时，需要用样本估计；这正是后续 MC、TD 和 Q-learning 等方法要解决的问题。

本章速记

- VI：直接反复应用 Bellman 最优算子，逼近 v_*。
- PI：精确评价当前策略，再对它的价值做贪心改进。
- 截断 PI：只评价有限步，是 VI 与 PI 之间的折中。
- 三者都需要模型，都是“展望一步 + 利用后续价值”的动态规划。
- 评价阶段回答“当前策略多好”，改进阶段回答“有没有更好的动作”。

5　L5：Monte Carlo 方法——没有模型时，用完整经历估计价值

本章要回答的问题

L4 的 VI/PI 都要知道环境模型 $p(s', r \mid s, a)$。如果模型不知道，还能不能做策略评价和策略改进？Monte Carlo 方法的答案是：可以，不直接算模型里的期望，而是用真实或模拟出来的 episode 回报做样本平均。

5.1　从“知道模型”到“只拿到样本”

在 L2–L4 中，很多更新都长这样：

$$\sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v(s') \right].$$

这表示：如果我们知道环境的转移概率和奖励分布，就能把所有可能的下一步结果按概率加权平均。问题是，真实任务里往往不知道这个模型。

Monte Carlo（MC）方法换了一个角度：既然价值本来就是**回报的期望**，那就直接多试几次，把实际观察到的回报取平均。

先抓直觉

模型方法像是在看“环境说明书”：知道每种结果的概率，于是直接算期望。MC 方法像是在做实验：不知道说明书，但可以反复采样，用大量实验结果的平均值逼近期望。

对动作价值来说，我们真正想估计的是

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

这句话的意思是：从状态 s 出发，第一步固定做动作 a，之后按照策略 π 行动，最终能得到多少折扣回报的平均值。

如果我们从同一个 (s, a) 出发，按 π 生成很多条 episode，每条 episode 都给出一个实际回报 $G^{(i)}(s, a)$。于是可以用

$$\boxed{\hat{q}_\pi(s, a) = \frac{1}{N} \sum_{i=1}^{N} G^{(i)}(s, a)}$$

来近似 $q_\pi(s, a)$。这里的帽子 $\hat{q}$ 提醒我们：它不是精确值，而是由样本得到的估计。

易错点

课件用均值估计引入 MC 时强调了 iid 样本的大数定律。在 RL 中，若多条 episode 是在相同策略下独立生成的，对应回报可以较自然地看成同分布样本；但同一条 episode 内不同访问位置通常相关。实际算法仍常用这些样本，直觉仍是“样本越多，均值越稳”，只是严格分析会比普通 iid 均值更细。

5.2 MC Basic：把策略迭代改成无模型版本

L4 的策略迭代有两步：**先评价当前策略，再根据价值改进策略**。MC Basic 保留这个框架，只把“评价”这一步从解 Bellman 方程改成采样估计。

MC Basic 的一轮

1. 固定当前策略 π_k；
2. 对每个状态–动作对 (s,a)，从这里开始采集足够多条 episode；
3. 每条 episode 后续都按 π_k 行动，记录它的回报；
4. 用这些回报的平均值估计 $q_{\pi_k}(s,a)$；
5. 对每个状态选择估计动作价值最大的动作，得到新策略 π_{k+1}。

策略改进时不再需要环境模型，只需要比较当前估计表：

$$a_k^*(s) \in \arg\max_a \hat{q}_k(s,a).$$

然后令新策略在状态 s 选择这个动作。

为什么 MC 更常估计 q 而不是 v？

如果模型已知，只知道 $v(s')$ 也能通过一步展望算出每个动作的好坏。但模型未知时，无法可靠计算 $\sum_{s',r} p(s',r \mid s,a)[r+\gamma v(s')]$。所以无模型控制通常直接估计动作价值 $q(s,a)$，因为它可以直接告诉我们“在状态 s 选哪个动作更好”。

5.3 更有效地使用一条 episode

MC Basic 的想法很清楚，但效率不高：如果每个 (s,a) 都单独生成很多 episode，采样成本会很大。实际上，一条 episode 里可能出现许多状态–动作对，每次出现都携带了从那一刻开始的后续回报信息。

设一条 episode 中从时间 t 开始的回报为

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots.$$

实际计算时常从 episode 末尾往前扫，用递推形式更新：

$$G \leftarrow R_{t+1} + \gamma G.$$

这样每扫到一个访问到的 (S_t, A_t)，就能得到一个对应的回报样本。

常见的使用方式有两种：

- **first-visit MC：**同一条 episode 中，一个 (s,a) 若出现多次，只用第一次访问后的回报；
- **every-visit MC：**每次访问 (s,a) 都使用，因此一条 episode 贡献更多样本。

课件中的 MC Exploring Starts 正是把这些想法合在一起：每产生一条 episode，就从后往前计算回报，并更新访问过的动作价值。这比“等某个 (s,a) 的所有 episode 收集完再更新”更像后续 RL 算法的工作方式。

广义策略迭代

很多 RL 算法都不再严格区分“先完全评价，再完全改进”。它们通常是一边让价值估计变好，一边让策略变好。这种评价与改进交替推进的思想，称为 generalized policy iteration（GPI）。

5.4 Exploring Starts：理论上好用，实践中难满足

MC 控制要想找到好策略，必须让每个动作都有机会被尝试。否则某个从未被试过的动作可能其实很好，但算法永远不知道。

Exploring starts 要求 episode 的初始状态–动作对可以覆盖所有 (s,a)，并且每一对都有机会被选为起点。这样做的好处是理论上干净：每个动作价值都能被充分估计。坏处也很明显：真实系统通常不能随意指定“从任意状态、任意动作开始”。

例如机器人、游戏或真实控制任务中，当前状态是环境演化出来的，不是我们想把它摆到哪里就摆到哪里。

5.5 ε-greedy：用软策略替代 exploring starts

为了不依赖 exploring starts，课件引入了**软策略（soft policy）**：在每个状态下，每个动作都有正概率被选择。最常见的软策略是 ε-greedy。

先找到当前估计下的贪心动作：

$$a^*(s) \in \arg\max_a \hat{q}(s,a).$$

然后大部分时间选它，小部分时间随机探索其他动作：

$$\pi(a \mid s) = \begin{cases} 1-\varepsilon+\dfrac{\varepsilon}{|\mathcal{A}(s)|}, & a = a^*(s), \\ \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & a \neq a^*(s). \end{cases}$$

这里 $|\mathcal{A}(s)|$ 是状态 s 下可选动作的数量。可以把它理解为：先拿出 ε 的概率做均匀随机探索，剩下 $1-\varepsilon$ 的概率交给当前最优动作。

探索与最优性不能同时免费获得

ε 大，探索能力强，但会经常故意不选当前最好的动作；ε 小，更接近贪心策略，但可能探索不足。固定 $\varepsilon > 0$ 时，算法通常是在固定的 ε-soft 策略集合中改进，不等于已经得到原问题的绝对最优确定性策略。实践中常使用逐渐减小的 ε，让早期多探索、后期更贪心。

5.6 L5 与前四讲的关系

问题	L4 的做法	L5 的做法
如何评价策略	用模型写出 Bellman 方程并求解	用回报样本均值估计 $q_\pi(s,a)$
如何改进策略	对模型计算出的动作价值贪心	对采样估计出的动作价值贪心或 ε-greedy
是否需要模型	需要 $p(s',r \mid s,a)$	不需要模型，但需要能采样
主要代价	计算模型期望或解方程	采样量大，且要保证足够探索

本章速记

- MC 的核心：价值是回报的期望，期望可以用样本平均估计。
- 无模型控制通常直接估计 $q(s,a)$，因为它能直接比较动作。
- MC Basic 是“策略迭代的无模型版本”：采样评价 q，再贪心改进。
- first-visit/every-visit 是更充分使用 episode 数据的方式。
- exploring starts 保证每个 (s,a) 被探索，但实践中很难满足。
- ε-greedy 用持续随机探索替代随意指定起点。

6　L6：随机逼近——为什么样本更新可以逐渐靠谱

本章要回答的问题

L5 用样本平均估计价值，但实际算法通常不会等所有样本都收集完再统一计算。它们更常见的形式是：来一个样本，就把当前估计往样本方向推一点。L6 解释这种随机、增量式更新背后的数学思想：随机逼近、Robbins–Monro 算法和随机梯度下降。

6.1　从普通平均到增量平均

假设我们想估计随机变量 X 的均值 $\mathbb{E}[X]$。最直接的办法是先收集 N 个样本，再计算

$$\bar{x}_N = \frac{1}{N}\sum_{i=1}^{N} x_i.$$

但在 RL 中，样本通常是不断到来的。我们希望每来一个样本就更新一次估计，而不是等到最后。

设 w_k 表示当前对均值的估计。新样本 x_k 到来时，一种非常重要的更新形式是

$$\boxed{w_{k+1} = w_k + \alpha_k(x_k - w_k)}$$

其中 $\alpha_k > 0$ 是步长。

这条式子最好这样读：

- $x_k - w_k$：新样本和当前估计之间的差，表示“这次样本认为你偏了多少”；
- α_k：相信这次样本多少；
- w_{k+1}：把旧估计朝新样本方向移动一小步。

当 $\alpha_k = 1/k$ 时，这个递推正好等价于普通样本平均。当 α_k 取其他满足条件的序列时，它仍可能收敛，这就进入了随机逼近的范围。

先抓直觉

后面很多 RL 更新都长得像

$$\text{新估计} = \text{旧估计} + \alpha \times (\text{目标} - \text{旧估计}).$$

L6 的作用，就是先用均值估计和 SGD 解释这个模板为什么有道理。

6.2　Robbins–Monro：只看到带噪声的函数值，也能找根

随机逼近（stochastic approximation, SA）研究的是：目标函数或方程本身不知道精确表达式，只能通过带噪声的数据观察它，是否仍能通过迭代逼近正确答案。

Robbins–Monro（RM）算法考虑一个找根问题：

$$g(w) = 0.$$

如果 g 完全已知，可以用普通数值方法找根。但现在我们只能在 w_k 处观测到一个带噪声的值

$$\tilde{g}(w_k) = g(w_k) + \eta_k.$$

RM 的更新是

$$\boxed{w_{k+1} = w_k - \alpha_k \tilde{g}(w_k)}$$

也就是：把带噪声的观测值当作方向信号，用小步长不断修正当前估计。

为什么是减号？

在一维、g 单调递增的情况下，若当前 w_k 在根的右边，通常有 $g(w_k) > 0$，减去它会把 w_k 往左推；若 w_k 在根的左边，通常有 $g(w_k) < 0$，减去负数会把 w_k 往右推。噪声会让每一步不完美，但只要步长安排得好，长期趋势仍可能指向根。

6.3　步长条件：既要走得远，也要越来越稳

课件中的 RM 收敛定理可以先不记完整技术细节，但一定要记住三类条件：

1. **函数形状要合适**。在一维版本中，常要求 g 单调并且斜率有上下界，这样根不会乱跑，也不会太平或太陡。
2. **步长要“无限但平方有限”**。常见条件是

$$\sum_{k=1}^{\infty} \alpha_k = \infty, \qquad \sum_{k=1}^{\infty} \alpha_k^2 < \infty.$$

   第一条保证总共走得足够远，不会太早停住；第二条保证后期步子足够小，噪声影响会被逐渐压下去。

3. **噪声不能系统性误导**。通常要求噪声在给定过去信息时均值为 0，并且方差有限。

典型选择是 $\alpha_k = 1/k$。它满足上面两条步长条件：调和级数发散，而平方和收敛。

易错点

随机逼近的收敛不是“只要用了样本更新就一定收敛”。步长、噪声、样本生成方式、目标函数形状都会影响结论。这点在后面看 TD、Q-learning、函数逼近时尤其重要。

6.4　均值估计其实是 RM 的特例

现在回到最熟悉的均值估计。我们想找到 $w = \mathbb{E}[X]$。这可以写成一个找根问题：

$$g(w) = w - \mathbb{E}[X] = 0.$$

但 $\mathbb{E}[X]$ 不知道，我们只能看到样本 x_k。于是用

$$\tilde{g}(w_k) = w_k - x_k$$

作为 $g(w_k)$ 的带噪声观测。代入 RM 更新：

$$w_{k+1} = w_k - \alpha_k(w_k - x_k) = w_k + \alpha_k(x_k - w_k).$$

这正是前面的增量均值更新。

所以，L6 的一个核心观点是：**样本均值不是孤立技巧，而是随机逼近理论中的一个简单例子**。这也解释了为什么 L5 中“用样本回报估计价值”的做法可以和迭代更新自然结合。

6.5　SGD：用一个样本梯度近似真实梯度

随机梯度下降（stochastic gradient descent, SGD）处理的是优化问题：

$$\min_w J(w) = \mathbb{E}[f(w, X)].$$

如果知道真实梯度，就可以做普通梯度下降：

$$w_{k+1} = w_k - \alpha_k \nabla J(w_k).$$

但真实梯度通常也是一个期望：

$$\nabla J(w_k) = \mathbb{E}[\nabla_w f(w_k, X)].$$

当这个期望难以精确计算时，SGD 用一个样本 x_k 的梯度来替代它：

$$\boxed{w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k)}.$$

SGD 为什么也能放进 RM 框架？因为最优化的一阶条件是

$$\nabla J(w) = 0.$$

也就是说，优化可以看作“寻找梯度为零的点”。样本梯度 $\nabla_w f(w_k, x_k)$ 就是对真实梯度 $\nabla J(w_k)$ 的带噪声观测。

均值估计也是 SGD

若取

$$f(w, X) = \frac{1}{2}\|w - X\|^2,$$

则样本梯度是 $w - X$。SGD 更新变成

$$w_{k+1} = w_k - \alpha_k(w_k - x_k) = w_k + \alpha_k(x_k - w_k),$$

又回到了增量均值估计。

6.6　BGD、MBGD 与 SGD 的区别

三者的区别主要在于：每次更新时用多少样本估计梯度。

方法	每轮用的样本	优点	代价
BGD	全部样本	梯度估计最稳定	每轮计算贵，数据大时不灵活
SGD	一个样本	每轮便宜，能在线更新	随机性大，轨迹会抖动
MBGD	一小批样本	在稳定性和效率之间折中	需要选择 batch 大小

SGD 的典型现象是：离最优点很远时，真实梯度信号通常较强，随机误差的相对影响较小，所以能较快靠近正确区域；离最优点很近时，真实梯度变小，样本噪声显得更明显，于是更新轨迹会在附近抖动。递减步长的作用，就是让这种后期抖动逐渐变小。

6.7　L6 对后续 RL 的意义

L6 不只是单独讲优化。它是在为后面的 TD 学习、Q-learning、actor–critic 等算法铺垫一种反复出现的模板：

$$\text{参数或价值的新估计} = \text{旧估计} + \alpha_k\,(\text{带噪声的目标信号} - \text{旧估计}).$$

在 L5 中，目标信号来自一整条 episode 的实际回报；在 TD 中，目标信号会变成“一步奖励 $+\gamma$ 下一状态估计价值”。因此，理解 L6 能帮助你看懂后面算法为什么看起来都像在做“误差修正”。

本章速记

- 增量均值更新：旧估计朝新样本方向移动一小步。
- RM 算法：只观察到带噪声的 $g(w)$，仍尝试求 $g(w) = 0$。
- 步长要既不太快消失，也要最终足够小：$\sum \alpha_k = \infty$，$\sum \alpha_k^2 < \infty$。
- 均值估计是 RM 的特例，也是 SGD 的特例。
- SGD 用样本梯度替代真实期望梯度，因此每步便宜但带随机性。
- L6 的思想会在 TD、Q-learning 等后续算法中反复出现。

7　L7：时序差分学习——不等 episode 结束就更新

本章要回答的问题

MC 必须等一条 episode 结束后，才能用完整回报更新价值。能否只观察一步转移，就立刻更新？怎样从状态价值过渡到 Sarsa、n-step Sarsa 和 Q-learning？为什么 Sarsa 是 on-policy，而 Q-learning 可以 off-policy?

7.1　先看全章地图：L7 是 L2、L5、L6 的交汇点

TD 学习和 MC 一样**不需要环境模型**，但它不再等待完整回报，而是把 Bellman 方程右侧的期望换成当前样本，再按 L6 的随机逼近形式更新：

Bellman 方程给出应满足的固定点

⇓

用一步或多步经验构造带噪声的 TD 目标

⇓

旧估计朝 TD 目标移动一小步

因此，L7 不是突然出现的一组新公式，而是把前面三条线接在了一起：

- **来自 L2：**价值函数满足 Bellman 方程；
- **来自 L5：**模型未知时，用实际经验样本代替模型期望；
- **来自 L6：**用“新估计 = 旧估计 + 步长 × 修正信号”逐步逼近方程的解。

TD 最核心的变化

MC 的目标是从当前时刻一直累加到 episode 结束的真实回报；一步 TD 的目标只看

实际得到的一步奖励 $+\gamma\times$ 自己对下一状态价值的当前估计.

也就是说，TD 用一个尚未完全学准的估计去更新另一个估计。这种做法叫作 **bootstrapping（自举）**。

7.2　基础 TD：用一步样本估计状态价值

7.2.1　从 Bellman 期望方程得到 TD 目标

给定策略 π，目标是估计所有状态的 $v_\pi(s)$。根据 Bellman 期望方程，

$$v_\pi(s)=\mathbb{E}[R_{t+1}+\gamma v_\pi(S_{t+1})\,|\,S_t=s]\,.$$

如果当前在 s_t，并实际观察到一步样本 (s_t,r_{t+1},s_{t+1})，就用

$$\boxed{\bar{v}_t=r_{t+1}+\gamma v_t(s_{t+1})}$$

作为右侧期望的一次样本近似。课本把 $\bar{v}_t$ 称为 **TD target（TD 目标）**。

于是，基础 TD 更新为

$$\boxed{v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\left[v_t(s_t) - \big(r_{t+1} + \gamma v_t(s_{t+1})\big)\right]}$$

并且

$$v_{t+1}(s) = v_t(s), \qquad s \neq s_t.$$

这第二行经常被省略，却很重要：**一次样本只更新本次访问到的状态**，没有访问到的状态保持不变。

7.2.2　把公式读成“旧值朝目标移动”

课本采用的 TD error 记号是

$$\boxed{\delta_t = v_t(s_t) - \bar{v}_t = v_t(s_t) - \big(r_{t+1} + \gamma v_t(s_{t+1})\big)}$$

所以更新式也可写成

$$v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\delta_t.$$

把符号展开，则是更直观的形式：

$$v_{t+1}(s_t) = v_t(s_t) + \alpha_t(s_t)\underbrace{\left[\bar{v}_t - v_t(s_t)\right]}_{\text{目标减当前估计}}.$$

若在这一次更新中暂时把 $\bar{v}_t$ 当作固定目标，且 $0 < \alpha_t(s_t) < 1$，那么

$$v_{t+1}(s_t) - \bar{v}_t = \big(1 - \alpha_t(s_t)\big)\left[v_t(s_t) - \bar{v}_t\right],$$

从而

$$\left|v_{t+1}(s_t) - \bar{v}_t\right| < \left|v_t(s_t) - \bar{v}_t\right|.$$

这正是“target”一词的数学含义：更新后的估计比更新前更靠近本次目标。

TD error 的正负号

本教材把 TD error 定义为“当前估计 − TD 目标”，所以更新写成 $-\alpha_t\delta_t$。读公式时应以本教材的定义为准；最不容易记错的办法，是直接读成

$$\text{新值} = \text{旧值} + \alpha_t(\text{目标} - \text{旧值}).$$

7.2.3　TD error 为什么能反映估计是否准确

当 $v_t = v_\pi$ 时，在给定 $S_t = s_t$ 的条件下，

$$\begin{aligned}\mathbb{E}[\delta_t \mid S_t = s_t] &= v_\pi(s_t) - \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s_t] \\ &= 0.\end{aligned}$$

第二个等号正是 Bellman 期望方程。因此，TD error 不只是 t 与 $t+1$ 两个时刻估计之间的差；它还是新经验 (s_t, r_{t+1}, s_{t+1}) 带来的 **innovation（新信息）**。TD 学习所做的事，就是用这份新信息修正当前估计。

7.3　TD 与 Monte Carlo：都无模型，但更新时机不同

比较角度	TD / Sarsa	Monte Carlo
何时更新	得到一步经验后即可立即更新，是增量式方法	必须等 episode 完整结束后，才能算折扣回报
适用任务	可用于 episodic task，也可用于没有终止状态的 continuing task	依赖有限长度 episode，适用于 episodic task
是否自举	是；目标中含下一状态或状态–动作对的当前估计	否；直接使用 episode 的实际折扣回报
是否依赖初值	自举需要价值的初始猜测，学习速度会受初值影响	完整回报不依赖下一状态的价值初值
估计方差	只涉及较少的随机变量，方差较低	完整回报包含许多随机奖励和状态，方差较高

不是“TD 永远优于 MC”

表中的差异来自目标的构造方式。TD 只看较短的随机片段，方差低，但会受尚未学准的自举值影响；MC 使用完整实际回报，不依赖自举初值，但整条轨迹带来更多随机性。后面的 n-step Sarsa 正是在这两个端点之间调节。

7.4　基础 TD 的收敛条件

对固定策略 π，课本给出的结论是：如果对每个 $s \in \mathcal{S}$ 都有

$$\sum_t \alpha_t(s) = \infty, \qquad \sum_t \alpha_t^2(s) < \infty,$$

那么基础 TD 的 $v_t(s)$ 几乎必然收敛到 $v_\pi(s)$。

这里 $\alpha_t(s) > 0$ 只发生在状态 s 于时刻 t 被访问时；否则把它看作 0。所以第一条条件还隐含了：**每个需要估计的状态必须被访问无穷多次，或至少足够多次**。这要依靠 exploring starts 或具有足够探索能力的策略。

课本也说明，实践中常用一个很小的常数步长。这时平方和条件不再成立，但仍可讨论期望意义下的收敛。在控制任务中，待评价的策略还会不断变化；常数步长能让价值估计继续跟踪这个非平稳目标，而逐渐过小的步长可能来不及适应新策略。

收敛结论有前提

“TD 会收敛”不是无条件命题。这里的定理针对**给定策略的价值评价**，并要求逐状态满足步长条件和充分访问。把 TD 与策略改进组合成控制算法后，策略本身在变化，不能把固定策略的定理不加区分地直接套用。

7.5　Sarsa：把状态价值 TD 改成动作价值 TD

7.5.1　为什么要从 v_π 换成 q_π

基础 TD 只能估计 $v_\pi(s)$。要直接比较同一状态下不同动作的好坏，需要动作价值 $q_\pi(s,a)$。它满足动作价值形式的 Bellman 方程：

$$\boxed{q_\pi(s,a)=\mathbb{E}[R_{t+1}+\gamma q_\pi(S_{t+1},A_{t+1})\,|\,S_t=s,A_t=a]}.$$

只要把基础 TD 中的“状态”换成“状态–动作对”，就得到 Sarsa。

7.5.2　Sarsa 更新与名字的来源

对按策略 π 产生的经验

$$(s_t,a_t,r_{t+1},s_{t+1},a_{t+1}),$$

Sarsa 的目标和更新分别是

$$\bar{q}_t^{\text{Sarsa}}=r_{t+1}+\gamma q_t(s_{t+1},a_{t+1}),$$

$$\boxed{q_{t+1}(s_t,a_t)=q_t(s_t,a_t)-\alpha_t(s_t,a_t)\left[q_t(s_t,a_t)-\bar{q}_t^{\text{Sarsa}}\right]}.$$

其余 $(s,a)\neq(s_t,a_t)$ 的动作价值保持不变。

算法每次需要的五个量依次是

$$\underbrace{s_t}_{\text{State}}\ \underbrace{a_t}_{\text{Action}}\ \underbrace{r_{t+1}}_{\text{Reward}}\ \underbrace{s_{t+1}}_{\text{State}}\ \underbrace{a_{t+1}}_{\text{Action}},$$

因此得名 **Sarsa**。

从数学上看，Sarsa 是求解给定策略动作价值 Bellman 方程的随机逼近算法。对固定 π，若每个 (s,a) 都满足

$$\sum_t\alpha_t(s,a)=\infty,\qquad\sum_t\alpha_t^2(s,a)<\infty,$$

也就是每个状态–动作对被充分访问，则课本给出的定理说明 $q_t(s,a)$ 几乎必然收敛到 $q_\pi(s,a)$。

7.5.3　怎样用 Sarsa 学策略：评价和改进交替进行

仅有上面的 Sarsa 更新，只能评价一个给定策略。为了搜索更好的策略，课本把它与 ε-greedy 策略改进结合起来：

1. 在 s_t 按当前 ε-greedy 策略选择 a_t；
2. 与环境交互，得到 r_{t+1},s_{t+1}；
3. 在 s_{t+1} 仍按当前策略选择 a_{t+1}；
4. 用五元组 $(s_t,a_t,r_{t+1},s_{t+1},a_{t+1})$ 更新 $q(s_t,a_t)$；
5. 根据新的动作价值，把 s_t 处的策略更新为 ε-greedy，然后继续下一步。

这里没有“把旧策略完全评价准确后才改进”。价值更新一步，策略也随即改进一步，这正是 L5 已出现的 **generalized policy iteration（广义策略迭代）**。使用 ε-greedy 的原因是：同一个策略还要负责生成后续经验，必须保留探索，才能让各个动作有机会被估计。

Sarsa 的目标包含“下一步真的会做什么”

Sarsa 的目标使用 $q_t(s_{t+1}, a_{t+1})$，而 a_{t+1} 是当前行为策略实际选出的动作。若策略会以 ε 的概率探索，这个探索动作也会进入更新目标。因此，Sarsa 评价的正是正在产生数据、也正在被改进的那个策略。

7.6　Expected Sarsa：对下一动作先取策略期望

Sarsa 在 s_{t+1} 只抽取一个实际动作 a_{t+1}。Expected Sarsa 则把下一动作的随机性直接平均掉：

$$\mathbb{E}[q_t(s_{t+1}, A)] = \sum_a \pi_t(a \mid s_{t+1}) q_t(s_{t+1}, a).$$

它的 TD 目标是

$$\boxed{\bar{q}_t^{\text{ExpSarsa}} = r_{t+1} + \gamma \sum_a \pi_t(a \mid s_{t+1}) q_t(s_{t+1}, a)}$$

并仍按

$$q_{t+1}(s_t, a_t) = q_t(s_t, a_t) + \alpha_t \big[\bar{q}_t^{\text{ExpSarsa}} - q_t(s_t, a_t)\big]$$

更新。

与 Sarsa 相比，它需要多做一次对动作的求和，计算量略增；但所涉及的随机变量从

$$\{S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}\} \quad \text{减少为} \quad \{S_t, A_t, R_{t+1}, S_{t+1}\},$$

所以课本指出它可以降低估计方差。

7.7　n-step Sarsa：在一步 TD 与 MC 之间调节

7.7.1　同一个回报可以在不同位置截断

动作价值定义为

$$q_\pi(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a].$$

完整回报可以在第 n 步截断，并把剩余部分换成动作价值：

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n q_\pi(S_{t+n}, A_{t+n}).$$

用实际样本和当前估计替代其中的随机量与真实价值，就得到

$$\boxed{\bar{q}_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n q_t(s_{t+n}, a_{t+n})}.$$

概念上的更新仍然是

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha \big[\bar{q}_t^{(n)} - q(s_t, a_t)\big].$$

7.7.2　为什么更新要延迟 n 步

在时刻 t 还没有观察到 $(r_{t+n}, s_{t+n}, a_{t+n})$，所以必须等到时刻 $t+n$，才能回头更新 (s_t, a_t)。按课本的时间下标，完整写法是

$$q_{t+n}(s_t,a_t) = q_{t+n-1}(s_t,a_t) - \alpha_{t+n-1}(s_t,a_t)\left[q_{t+n-1}(s_t,a_t) - \left(\sum_{k=0}^{n-1}\gamma^k r_{t+k+1} + \gamma^n q_{t+n-1}(s_{t+n},a_{t+n})\right)\right].$$

7.7.3　n 控制了什么

取值	得到的方法	目标与性质
$n=1$	一步 Sarsa	$r_{t+1}+\gamma q(s_{t+1},a_{t+1})$；等待短、方差较低，但更依赖自举初值
一般 n	n-step Sarsa	使用前 n 个实际奖励，再从第 n 步动作价值自举
$n\to\infty$	Monte Carlo	一直使用实际奖励到 episode 结束；课本在统一表达中取 $\alpha=1$，得到完整回报样本

n 较小：更接近 Sarsa，偏差较大、方差较低

n 较大：更接近 MC，偏差较小、方差较高

课本把小 n 时较大的偏差归因于对初始价值猜测的依赖。这里介绍的 n-step Sarsa 本身仍是策略评价方法；要学习更好的策略，仍需像一步 Sarsa 一样加入策略改进。

7.8　Q-learning：直接逼近最优动作价值

7.8.1　把“下一实际动作”换成“下一状态最大动作价值”

Sarsa 求解给定策略的 Bellman 方程。Q-learning 改为求解动作价值形式的 Bellman 最优方程：

$$\boxed{q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q(S_{t+1},a') \,\middle|\, S_t=s, A_t=a\right].}$$

因此它的 TD 目标是

$$\bar{q}_t^{Q} = r_{t+1} + \gamma \max_{a'} q_t(s_{t+1},a'),$$

更新式为

$$\boxed{q_{t+1}(s_t,a_t) = q_t(s_t,a_t) + \alpha_t(s_t,a_t)\left[r_{t+1} + \gamma \max_{a'} q_t(s_{t+1},a') - q_t(s_t,a_t)\right].}$$

同样，只有本次访问的 (s_t, a_t) 被更新。

Q-learning 给定 (s_t, a_t) 后，只需要 (r_{t+1}, s_{t+1})；Sarsa 还必须知道行为策略在下一状态实际选择的 a_{t+1}。这一个差别正对应两个算法不同的数学目标。

比较	Sarsa	Q-learning
TD 目标	$r_{t+1}+\gamma q_t(s_{t+1},a_{t+1})$	$r_{t+1}+\gamma \max_{a'} q_t(s_{t+1},a')$
解什么方程	给定策略的 Bellman 方程	Bellman 最优方程
直接估计什么	$q_\pi(s,a)$	最优动作价值
下一动作是否进入样本	需要行为策略实际选出的 a_{t+1}	不需要；更新内部直接取最大值
策略关系	行为策略同时也是被评价、被改进的目标策略	产生动作的行为策略可以不同于由 q 得到的目标策略

7.8.2　为什么仍然需要探索

“行为策略可以是任意策略”不等于“数据覆盖可以任意差”。课本的离策略实验表明：均匀行为策略能频繁访问各状态–动作对，Q-learning 的误差下降较快；当行为策略探索性减弱时，学习速度明显下降。原因是未被充分访问的动作价值没有足够样本可更新。

Q-learning 也会自举，所以初始 $q_0(s,a)$ 会影响收敛速度。课本实验中，初值越接近真实最优价值，收敛越快；但即使初值不准确，只要经验和探索充分，估计仍可逐步改进。

7.9　On-policy、off-policy、online、offline 不要混

7.9.1　先区分两种“策略”

- **行为策略（behavior policy）π_b：**真正与环境交互、生成动作和经验样本的策略；
- **目标策略（target policy）π_T：**正在被评价或不断改进、希望最终学到的策略。

  如果行为策略与目标策略相同，称为 **on-policy**；如果两者不同，称为 **off-policy**。

- **Sarsa 是 on-policy：** a_t 和 a_{t+1} 都由当前策略产生，而 Sarsa 又在评价并改进这个策略；
- **本章的 MC 控制是 on-policy：**被评价和改进的策略就是产生 episode 的策略；
- **Q-learning 具有 off-policy 性质：**它求 Bellman 最优方程，更新目标中的最大化不依赖 π_b 在 s_{t+1} 实际会选什么。因而可以用探索性行为策略收集数据，同时把由当前最优动作价值导出的贪心策略作为目标策略。

Q-learning 的“两条线”

$$\underbrace{\pi_b:\ s_t \longrightarrow a_t \longrightarrow (r_{t+1},s_{t+1})}_{\text{负责产生数据}} \qquad \underbrace{\pi_T(s)=\arg\max_a q(s,a)}_{\text{负责表示要学的策略}}.$$

数据由谁产生和更新最终指向谁，可以是两件事。这正是 off-policy 学习能利用其他策略、甚至预先收集的数据的原因。

on/off-policy 讨论的是**产生数据的策略与学习目标的策略是否相同**；online/offline 讨论的是**数据在学习时是否正在与环境交互产生**：

概念对	判断问题	本章中的含义
on-policy / off-policy	行为策略和目标策略是否相同	Sarsa、MC 是 on-policy；Q-learning 可使用不同的行为与目标策略
online / offline	是否边与环境交互边更新	online 边采样边学；offline 使用预先收集的经验，不在学习时与环境交互

课本同时给出了 Q-learning 的在线实现和使用预先收集 episode 的离线实现。作为 off-policy 方法，Q-learning 允许行为策略和目标策略不同；实际使用时仍必须保证行为策略能提供充分经验。

三个常见误解

- off-policy 不表示“不需要策略”，而是行为策略与目标策略可以不同；
- offline 不表示 off-policy；前者描述数据何时收集，后者描述两种策略的关系；
- Q-learning 的 max 只决定 TD 目标，当前动作 a_t 仍由行为策略产生，仍需探索。

7.10　统一视角：算法的骨架相同，只是 TD 目标不同

对动作价值学习，本章算法都可写成

$$\boxed{q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - \bar{q}_t\right]}$$

或等价地写成

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) + \alpha_t(s_t,a_t)\left[\bar{q}_t - q_t(s_t,a_t)\right].$$

真正区分算法的是 $\bar{q}_t$ 以及它对应的方程：

算法	TD 目标 $\bar{q}_t$	求解对象
Sarsa	$r_{t+1} + \gamma q_t(s_{t+1},a_{t+1})$	给定策略的 Bellman 方程
n-step Sarsa	$\sum_{k=0}^{n-1}\gamma^k r_{t+k+1} + \gamma^n q_t(s_{t+n},a_{t+n})$	给定策略的 n 步 Bellman 形式
Q-learning	$r_{t+1} + \gamma \max_a q_t(s_{t+1},a)$	Bellman 最优方程
Monte Carlo	$r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots$	用完整回报表达的 Bellman 方程

统一地说，这些方法都在用随机逼近求解

$$q(s,a) = \mathbb{E}[\bar{q}_t \mid S_t = s, A_t = a].$$

除 Q-learning 外，表中其他方法求的是给定策略的 Bellman 方程；Q-learning 求的是 Bellman 最优方程。

7.11　拿到一条经验时，怎样判断该用哪个目标

假设手中已有 $(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$，只看下一部分如何处理：

- 要评价当前实际执行的策略：用 $q_t(s_{t+1}, a_{t+1})$，得到 Sarsa；
- 要减少下一动作采样带来的随机性：用 $\sum_a \pi_t(a \mid s_{t+1}) q_t(s_{t+1}, a)$，得到 Expected Sarsa；
- 愿意等待 n 步，用更多真实奖励：构造 n-step Sarsa 目标；
- 要直接逼近最优动作价值：用 $\max_a q_t(s_{t+1}, a)$，得到 Q-learning；
- 一直等到 episode 结束、完全不用自举值：使用完整回报，得到 MC。

L7 本章速记

- TD = Bellman 方程 + 经验样本 + 随机逼近。
- 基础 TD：$v(s_t) \leftarrow v(s_t) + \alpha[r_{t+1} + \gamma v(s_{t+1}) - v(s_t)]$。
- TD 可一步后立即更新、能处理 continuing task，并且会 bootstrapping。
- Sarsa 用实际下一动作：$r + \gamma q(s', a')$，评价产生数据的策略。
- Expected Sarsa 对下一动作取策略期望，可降低由 A_{t+1} 带来的方差。
- n-step Sarsa 在一步 Sarsa 与完整回报 MC 之间连接偏差和方差。
- Q-learning 用 $r + \gamma \max_{a'} q(s', a')$，直接求 Bellman 最优方程。
- Sarsa、MC 是 on-policy；Q-learning 允许行为策略与目标策略不同。
- off-policy 不等于不探索；没有充分访问，就没有足够样本更新。
- 所有方法都能读成："旧估计朝本算法定义的目标移动一小步"。

8 L8：Value Function Methods——从表格走向函数

前七讲默认用表格保存价值：每个状态对应一个 $V(s)$，每个状态–动作对对应一个 $Q(s,a)$。这种表示清楚、定理也容易分析，但当状态连续或数量极大时，表格既存不下，也无法给从未访问过的状态估值。L8 的核心变化是用带参数的函数代替表格：

$$v_\pi(s) \approx \hat{v}(s,w), \qquad q_\pi(s,a) \approx \hat{q}(s,a,w),$$

其中 $w \in \mathbb{R}^m$ 是需要从样本中学习的参数。

L8 的一句话主线

把 L7 的“只修改一个表项”改成“沿函数梯度修改参数”：

$$w_{t+1} = w_t + \alpha_t \underbrace{(\text{target} - \text{当前估计})}_{\text{TD error}} \underbrace{\nabla_w \hat{v} \text{ 或} \nabla_w \hat{q}}_{\text{把误差传给参数}}.$$

target 决定算法在逼近哪个 Bellman 方程；梯度决定一次误差怎样影响整个函数。

8.1 为什么表格不够：存储、查询、更新与泛化

比较维度	表格表示	函数表示
存储	保存 $\lvert\mathcal{S}\rvert$ 或 $\lvert\mathcal{S}\rvert\lvert\mathcal{A}\rvert$ 个数	只保存 m 个参数，通常 m 远小于表项数
查询	按索引读取一个表项	做一次前向计算 $\hat{v}(s,w)$ 或 $\hat{q}(s,a,w)$
更新	通常只改变刚访问的表项	修改 w，因而可能同时改变许多状态的估值
未见状态	没有样本的表项无法自行推断	相似特征可共享参数，产生**泛化**
代价	精确表示有限表格，但规模受限	节省存储并可泛化，但产生**近似误差**

直观理解

函数近似相当于把许多状态“绑”在同一组参数上。一次更新不仅修正 S_t，也会改变与 S_t 使用相似特征的状态。泛化正是由此产生；相互干扰也由此产生。

8.2 三类常见逼近器：线性、特征展开与神经网络

最重要的线性形式是

$$\hat{v}(s,w) = \phi(s)^\top w, \qquad \hat{q}(s,a,w) = \phi(s,a)^\top w,$$

其中 ϕ 是预先设计的特征。这里的“线性”是指对参数 w 线性；$\phi(s)$ 本身完全可以包含 s^2、Fourier 基、tile coding 等非线性变换。神经网络则直接用非线性函数表示 $\hat{v}$ 或 $\hat{q}$，表达能力更强，但理论分析与训练稳定性更复杂。

表格其实是线性函数近似的特例

若有 n 个状态，令 $\phi(s_i) = e_i$ 为第 i 个单位向量，则

$$\hat{v}(s_i, w) = e_i^\top w = w_i.$$

参数向量 w 就是原来的价值表。因此 L8 不是抛弃表格方法，而是把它包含在更一般的表示中。

如果所有真实状态价值已经知道，把状态逐行组成设计矩阵 Φ，普通最小二乘解为

$$w_{\mathrm{LS}} = (\Phi^\top \Phi)^{-1} \Phi^\top v_\pi,$$

前提是 $\Phi^\top \Phi$ 可逆。RL 的困难恰恰是 v_π 通常未知，只能从交互样本构造学习目标。

8.3 先把目标函数说清楚：状态分布决定“哪里更重要”

固定策略 π 时，一个自然目标是加权价值均方误差

$$J_E(w) = \mathbb{E}_{S\sim d}\big[(v_\pi(S) - \hat{v}(S, w))^2\big] = \|v_\pi - \Phi w\|_D^2,$$

其中 $D = \mathrm{diag}(d)$，且 $\|x\|_D^2 = x^\top D x$。分布 d 不是装饰：它决定哪些状态的误差在总目标中权重更大。

对课程重点的 on-policy continuing setting，常用策略 π 的平稳分布 d_π：

$$d_\pi^\top = d_\pi^\top P_\pi, \qquad \sum_s d_\pi(s) = 1.$$

直观上，长期运行策略后经常访问的状态会得到较大权重。该解释依赖马尔可夫链具有合适的不可约性、遍历性等条件；不能在任意链上无条件假定唯一平稳分布存在。

“最小误差”必须先说明按什么分布

同一逼近器在均匀分布下的最优参数，未必是在 d_π 下的最优参数。因此比较两个函数逼近器时，必须同时说明特征、数据分布和误差度量。

8.4 从监督式 SGD 到半梯度 TD

若能观察到标签 $v_\pi(S_t)$，对 J_E 做随机梯度下降可写成

$$w_{t+1} = w_t + \alpha_t [v_\pi(S_t) - \hat{v}(S_t, w_t)] \nabla_w \hat{v}(S_t, w_t).$$

但真实价值未知。教材依次给出两种可采样替代：

- 用完整回报 G_t 替代，得到带函数近似的 Monte Carlo 更新；
- 用一步 TD target $R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t)$ 替代，得到

$$\boxed{w_{t+1} = w_t + \alpha_t \delta_t \nabla_w \hat{v}(S_t, w_t)}, \qquad \delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t).$$

为什么叫 semi-gradient（半梯度）

TD target 中的 $\hat{v}(S_{t+1}, w_t)$ 也依赖 w_t，但更新只对当前估计 $\hat{v}(S_t, w_t)$ 求梯度，把 target 暂时当成常数。因此它不是对“单样本 TD error 平方”做完整梯度下降，也不能据此宣称它直接最小化 J_E。教材随后用投影 Bellman 方程给出严格解释。

一次线性 TD 更新

设 $\phi(S_t) = (1,2)^\top$，$\phi(S_{t+1}) = (1,1)^\top$，$w_t = (0.5,1)^\top$，$R_{t+1} = 1$，$\gamma = 0.9$，$\alpha = 0.1$。则

$$\hat{v}(S_t, w_t) = 2.5, \qquad \hat{v}(S_{t+1}, w_t) = 1.5,$$

$$\delta_t = 1 + 0.9 \times 1.5 - 2.5 = -0.15, \qquad w_{t+1} = w_t + 0.1(-0.15)(1,2)^\top = (0.485, 0.970)^\top.$$

一次样本同时改变两个参数，之后所有使用这两个参数的状态估值都可能改变。

8.5 TD-Linear：把梯度换成特征

在线性表示下 $\nabla_w \hat{v}(s,w) = \phi(s)$，于是

$$\boxed{w_{t+1} = w_t + \alpha_t \big[R_{t+1} + \gamma \phi(S_{t+1})^\top w_t - \phi(S_t)^\top w_t\big]\phi(S_t)}.$$

这就是课程中的 TD-Linear。它保留 L7 的“一步采样加自举”，只是把“改一个表项”换成“沿当前状态的特征方向改参数”。若 S_{t+1} 为终止状态，约定其 bootstrap 价值为零。

8.6 TD-Linear 真正在解什么：投影 Bellman 方程

令

$$T_\pi v = r_\pi + \gamma P_\pi v, \qquad \Pi_D = \Phi(\Phi^\top D \Phi)^{-1} \Phi^\top D,$$

其中 Π_D 把任意价值向量按 D-加权距离投影到特征子空间 $\{\Phi w : w \in \mathbb{R}^m\}$。TD-Linear 的极限满足

$$\boxed{\Phi w^* = \Pi_D T_\pi(\Phi w^*)},$$

即“做一次 Bellman 更新，再投影回函数类”后的固定点。

教材比较了三个容易混淆的目标：

$$\begin{aligned} J_E(w) &= \|\Phi w - v_\pi\|_D^2, && \text{价值近似误差,} \\ J_{BE}(w) &= \|\Phi w - T_\pi(\Phi w)\|_D^2, && \text{Bellman error,} \\ J_{PBE}(w) &= \|\Phi w - \Pi_D T_\pi(\Phi w)\|_D^2, && \text{projected Bellman error.} \end{aligned}$$

TD-Linear 对应第三种投影固定点，而不是一般意义下直接最小化第一种目标。

直观理解

真实 Bellman 更新 $T_\pi(\Phi w)$ 往往已经离开特征子空间，现有函数类根本表示不了它。投影 Π_D 的作用是把它拉回可表示集合，所以 TD 最后解的是“Bellman 更新 + 投影”的固定点。

在 on-policy 平稳采样下，线性 TD 的期望更新可整理为

$$w_{t+1} = w_t + \alpha_t(b - A w_t),$$

$$A = \Phi^\top D(I - \gamma P_\pi)\Phi, \qquad b = \Phi^\top D r_\pi.$$

在教材给出的适当条件下，w_t 收敛到

$$w^* = A^{-1}b.$$

典型前提包括：线性特征具有足够秩、按目标策略的平稳分布采样、奖励有界，且步长满足随机逼近所需条件。还可得到近似误差界

$$\|\Phi w^* - v_\pi\|_D \le \frac{1}{1-\gamma} \min_w \|\Phi w - v_\pi\|_D.$$

不要把线性 on-policy 定理无限外推

上述结论针对固定策略、线性函数近似和相匹配的 on-policy 分布。它不自动证明非线性神经网络 TD、off-policy 评价或函数近似 Q-learning 一定收敛。对超出定理范围的算法，应区分“实践中常用”与“已有该定理保证”。

8.7 LSTD：直接估计线性方程

既然极限满足 $Aw = b$，Least-Squares TD（LSTD）的思路就是用样本累计

$$\widehat{A} = \sum_t \phi_t(\phi_t - \gamma\phi_{t+1})^\top, \qquad \widehat{b} = \sum_t R_{t+1}\phi_t,$$

再解 $\widehat{A}w = \widehat{b}$。它能充分复用样本，常比逐步小步长更新更快；代价是存储矩阵和求解线性系统的计算量更大，且这里讨论的是线性、固定策略状态价值评价。

8.8 带函数近似的 Sarsa：评价实际执行的下一动作

把 L7 的 Sarsa 表格更新换成参数更新：

$$\boxed{w_{t+1} = w_t + \alpha_t \delta_t^{\mathrm{Sar}} \nabla_w \hat{q}(S_t, A_t, w_t)},$$

$$\delta_t^{\mathrm{Sar}} = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, w_t) - \hat{q}(S_t, A_t, w_t).$$

若 $\hat{q}(s,a,w) = \phi(s,a)^\top w$，梯度就是 $\phi(S_t, A_t)$。控制时仍由当前 $\hat{q}$ 构造 ε-greedy 策略，并由该策略实际生成 A_{t+1}，因此标准 Sarsa 是 on-policy。

Sarsa 函数近似的最小记忆单元

$$\text{target} = R + \gamma \hat{q}(S', A', w), \quad \text{error} = \text{target} - \hat{q}(S, A, w), \quad w \leftarrow w + \alpha\, \text{error}\, \nabla_w \hat{q}(S, A, w).$$

结构与 L7 完全相同；新增的只是最后一个梯度因子。

8.9 带函数近似的 Q-learning：target 仍然取最大值

Q-learning 的参数更新为

$$\boxed{w_{t+1} = w_t + \alpha_t \delta_t^{Q} \nabla_w \hat{q}(S_t, A_t, w_t)},$$

$$\delta_t^{Q} = R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, w_t) - \hat{q}(S_t, A_t, w_t).$$

行为动作 A_t 可以由探索策略产生，但 target 评价的是下一状态的贪心动作，因此按标准 backup 分类它是 off-policy。

方法	target 的下一步部分	参数更新方向
Sarsa	$\hat{q}(S', A', w)$，A' 为实际采样动作	$\nabla_w \hat{q}(S, A, w)$
Q-learning	$\max_a \hat{q}(S', a, w)$	$\nabla_w \hat{q}(S, A, w)$

与 L7 的术语保持一致

课程材料会按“边采样边更新”或“先由行为策略收集经验再更新”描述不同实现版本；但只要 backup 使用 $\max_a \hat{q}(S', a, w)$，标准 RL 分类仍把它看作 off-policy target。online/offline 描述数据何时收集，on/off-policy 描述行为策略与学习目标是否一致，二者不是同一维度。

8.10 为什么普通深度 Q-learning 需要稳定化

若直接令神经网络参数 w 同时出现在当前预测和 target 中，可写出形式目标

$$\mathbb{E}\Big[\big(R + \gamma \max_a \hat{q}(S', a, w) - \hat{q}(S, A, w)\big)^2\Big].$$

此时网络每更新一次，预测和训练标签都会一起移动，样本又按轨迹连续出现，普通监督学习的“固定标签、近似独立样本”直觉不再成立。教材中的 DQN 用两项关键技术缓解这两个问题：

1. **target network**：暂时冻结生成 target 的参数；
2. **experience replay**：从经验池均匀随机抽取 mini-batch。

8.11 DQN 技术一：主网络与目标网络

主网络参数记为 w，目标网络参数记为 w_T。对一条经验 (s, a, r, s')，课程版本的 target 为

$$y_T = r + \gamma \max_{a'} \hat{q}(s', a', w_T),$$

然后只更新主网络，使

$$L(w) = \big(y_T - \hat{q}(s, a, w)\big)^2$$

减小。每隔 C 次主网络更新，再复制 $w_T \leftarrow w$。若 s' 为终止状态，应令 bootstrap 项为零；代码中常写成

$$y_T = r + \gamma(1 - \texttt{done}) \max_{a'} \hat{q}(s', a', w_T).$$

直观理解

冻结 w_T 的一段时间内，y_T 对主网络参数 w 是固定标签，梯度只需穿过 $\hat{q}(s, a, w)$。定期复制让标签跟随学习进度，但不会在每个微小更新中立即追着主网络移动。

8.12 DQN 技术二：经验回放

把行为策略生成的经验存入 replay buffer

$$\mathcal{B} = \{(s, a, r, s')\},$$

训练时不按收集顺序使用，而是从 $\mathcal{B}$ 中均匀随机抽取 mini-batch。这样做有三层作用：

- 打散相邻轨迹样本的强时间相关性；
- 同一经验可被多次利用，提高样本利用率；
- 让 mini-batch 梯度更接近课程目标中对经验分布的平均。

经验通常由带探索的行为策略产生，target 却取贪心最大值，所以 DQN 是 off-policy 方法。

经验回放不是“数据随便放进去就行”

经验池仍需覆盖学习任务所需的状态–动作区域。若行为策略从未访问关键动作，随机抽样不能凭空创造信息。课程算法强调均匀回放；其他采样方案需要相应修正与额外分析，不应与本章基础版本混写。

8.13 DQN 的网络输出、训练循环与损失解释

离散动作下常见两种网络接口：

- 输入 (s, a)，输出一个标量 $\hat{q}(s, a, w)$；
- 只输入 s，一次输出所有离散动作的 Q 值向量，便于计算 $\arg\max_a$。

DQN 基础循环

1. 用带探索的行为策略与环境交互，把 $(S, A, R, S', \texttt{done})$ 放入经验池；
2. 均匀抽取 mini-batch；
3. 用目标网络计算每条样本的 y_T，终止样本不 bootstrap；
4. 用 mini-batch 降低 $(y_T - \hat{q}(S, A, w))^2$，只更新主网络；
5. 每隔 C 次更新复制 $w_T \leftarrow w$。

训练损失很小，不代表价值函数一定正确

损失只检查经验池中出现的输入以及由当前 target network 生成的标签。如果数据覆盖不足、target 本身偏离真实最优价值，网络完全可能把训练损失降得很低，却仍在未覆盖状态上严重出错。因此损失曲线必须结合回报、覆盖范围和独立评估理解。

8.14 L7 到 L8：只看 target 与更新对象就不会迷路

方法	target	实际被更新的对象
表格 TD	$R + \gamma V(S')$	当前表项 $V(S)$
线性 TD	$R + \gamma \phi(S')^\top w$	参数 w，方向为 $\phi(S)$
Sarsa–FA	$R + \gamma \hat{q}(S', A', w)$	参数 w，方向为 $\nabla_w \hat{q}(S, A, w)$

方法	target	实际被更新的对象
Q-learning–FA	$R+\gamma \max_a \hat{q}(S', a, w)$	参数 w，方向为 $\nabla_w \hat{q}(S, A, w)$
DQN	$R+\gamma \max_a \hat{q}(S', a, w_T)$	主网络 w；w_T 周期性复制

核心结论

L8 的算法名称很多，但只有两次结构变化：

1. 表格变成 $\hat{v}/\hat{q}$ 后，更新末尾多出一个对参数的梯度；
2. DQN 再把 target 中的参数从 w 换成暂时冻结的 w_T，并用经验回放组成 mini-batch。

8.15 初学者最容易产生的八个误解

1. **“线性近似只能拟合状态的直线函数。”**
   线性是对 w 而言；非线性特征 $\phi(s)$ 仍可表达复杂形状。
2. **“函数近似只是更省内存，不会改变更新含义。”**
   参数共享带来泛化，也会让一次更新影响其他状态。
3. **“TD target 替代真实价值后，仍是在对 J_E 做普通 SGD。”**
   不准确。TD 是半梯度更新；线性理论对应投影 Bellman 固定点。
4. **“Bellman error 与 projected Bellman error 是同一个量。”**
   不是。后者先把 Bellman 更新投影回可表示的特征空间。
5. **“表格方法与线性函数近似互不相关。”**
   one-hot 特征使线性表示精确退化为表格。
6. **“线性 TD 的收敛定理也保证任意神经网络 Q-learning 收敛。”**
   定理范围不同，不能外推。
7. **“目标网络是第二个独立学习的目标。”**
   基础 DQN 中它不单独优化，而是周期性复制主网络参数。
8. **“DQN loss 接近零就证明学到了 q_*。”**
   loss 只说明对当前数据和当前 target 拟合得好，不证明数据覆盖与 target 正确。

8.16 自测题与参考答案

题目

1. 为什么状态分布 d 会改变函数近似的最优参数?
2. 写出半梯度 TD 的一般更新，并说明为什么叫“半梯度”。
3. 证明 one-hot 特征下 $\hat{v}(s, w) = \phi(s)^\top w$ 等价于价值表。
4. 写出投影 Bellman 固定点，并说明投影为什么必要。

5. 写出 Sarsa–FA 与 Q-learning–FA 的 target，指出唯一关键差异。
6. DQN 中主网络与目标网络各做什么？
7. 为什么经验回放不能弥补行为策略完全没有覆盖的动作？
8. 训练损失很小但平均回报很差，至少给出两种可能原因。

参考答案

1. J_E 对各状态误差按 $d(s)$ 加权；分布变化会改变不同状态之间的折中。
2. $w \leftarrow w + \alpha[R + \gamma\hat{v}(S', w) - \hat{v}(S, w)]\nabla_w \hat{v}(S, w)$；target 也依赖 w，但求导时把它暂当常数。
3. 令状态 s_i 的特征为单位向量 e_i，则 $e_i^\top w = w_i$，恰好读取第 i 个表项。
4. $\Phi w^* = \Pi_D T_\pi(\Phi w^*)$；Bellman 更新一般不在特征子空间内，必须投影回可表示集合。
5. Sarsa 用 $R + \gamma\hat{q}(S', A', w)$；Q-learning 用 $R + \gamma \max_a \hat{q}(S', a, w)$。差异是实际下一动作与最大动作价值。
6. 主网络通过梯度下降拟合 target；目标网络暂时冻结并生成 target，每隔 C 次从主网络复制参数。
7. 回放只能重采已有经验；经验池里没有的动作仍没有奖励与转移信息。
8. 可能是经验覆盖不足、target 偏差、只在训练样本上拟合、探索不足，或终止状态 bootstrap 处理错误等。

8.17 一页总结

核心结论

1. 函数近似用少量参数表示大量状态价值，获得泛化，同时引入近似误差与状态间干扰。
2. 线性表示是对参数线性；one-hot 特征使它包含表格表示。
3. 价值误差必须连同状态分布一起定义；on-policy 分析常使用策略平稳分布 d_π。
4. 半梯度 TD 更新是“TD error 乘当前估计对参数的梯度”。
5. TD-Linear 的严格目标是投影 Bellman 固定点，而非直接最小化普通价值 MSE。
6. 线性 TD 收敛结论有明确的固定策略、on-policy、特征与步长条件，不能无条件外推。
7. Sarsa–FA 使用实际下一动作，Q-learning–FA 使用下一状态最大动作价值。
8. DQN 用目标网络暂时固定标签，用经验回放打散相关性并复用样本。
9. 终止状态不 bootstrap；DQN 的低训练损失也不等于真实价值误差必然低。
10. 识别算法时先圈出 target，再看更新的是表项、线性参数还是神经网络参数。

掌握标准：能够从任意价值函数更新中指出“逼近器、target、数据分布、梯度方向和定理适用范围”，并能解释目标网络与经验回放各自解决的结构性问题。

9 L9：Policy Gradient Methods——直接优化策略

L1–L8 的主角一直是价值函数：先估计 v 或 q，再从价值中提取策略。L9 第一次把策略本身写成可微函数 $\pi(a \mid s, \theta)$，选择一个标量指标 $J(\theta)$，然后直接做梯度上升：

$$\theta_{t+1} = \theta_t + \alpha_t \nabla_\theta J(\theta_t).$$

L9 的三问

1. 用哪个标量指标定义“策略更好”？
2. 指标的梯度怎样写成可采样的期望？
3. 未知的 q_π 怎样用 Monte Carlo 样本替代？

答案依次是：平均价值/平均奖励、策略梯度定理、REINFORCE。

9.1 从策略表到策略函数

表格策略为每个 (s, a) 保存一个概率；函数策略用共享参数生成整个动作分布：

$$\pi(a \mid s, \theta), \qquad \sum_{a \in \mathcal{A}} \pi(a \mid s, \theta) = 1.$$

离散动作下，教材使用 softmax 参数化：

$$\boxed{\pi(a \mid s, \theta) = \frac{\exp(h(s, a, \theta))}{\sum_{a'} \exp(h(s, a', \theta))}}.$$

h 是动作偏好，可由线性函数或神经网络产生。softmax 保证每个动作概率为正且总和为一，因此 $\ln \pi(a \mid s, \theta)$ 有定义，策略也具有随机探索性。

直观理解

价值方法先问“每个动作值多少”，再选最大者；策略方法直接问“动作概率应怎样改变”。共享参数让相似状态的策略相互泛化，代价是优化目标通常非凸，不能仅凭梯度上升保证找到全局最优策略。

9.2 指标一：平均状态价值

给定状态权重分布 d，定义

$$\bar{v}_\pi = \sum_s d(s) v_\pi(s) = \mathbb{E}_{S \sim d}[v_\pi(S)].$$

教材区分两类 d：

- d_0 与策略无关，例如均匀分布，或只关心固定初始状态 s_0；此时记作 $\bar{v}_\pi^0$；
- d_π 是策略诱导的平稳分布，按长期访问频率给状态加权；此时常简记 $\bar{v}_\pi$。

在折扣任务中，从 $S_0 \sim d$ 出发的期望折扣回报满足

$$J(\theta) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t R_{t+1}\right] = \sum_s d(s) v_\pi(s).$$

9.3 指标二：平均一步奖励

令

$$r_\pi(s) = \sum_a \pi(a \mid s, \theta) r(s, a),$$

平均奖励定义为

$$\bar{r}_\pi = \sum_s d_\pi(s) r_\pi(s) = \mathbb{E}_{S \sim d_\pi}[r_\pi(S)].$$

它也等于长期时间平均奖励：

$$\bar{r}_\pi = \lim_{n \to \infty} \frac{1}{n} \mathbb{E}_\pi \left[\sum_{t=0}^{n-1} R_{t+1} \right],$$

这里需要教材关于平稳分布和长期极限的适当条件。

在折扣情形且平均价值按 d_π 加权时，教材给出

$$\bar{r}_\pi = (1 - \gamma) \bar{v}_\pi,$$

所以二者可被同时最大化。

三个指标不能只看名字混写

$\bar{v}_\pi^0$、$\bar{v}_\pi$ 与 $\bar{r}_\pi$ 的状态权重不同。策略梯度中的状态分布也会随指标与折扣/不折扣设定变化；不能把一个推导中的分布直接搬到所有场景。

9.4 策略梯度定理：最重要的统一表达

教材 Theorem 9.1 将多种指标下的结果统一写成

$$\boxed{\nabla_\theta J(\theta) = \sum_s \eta(s) \sum_a \nabla_\theta \pi(a \mid s, \theta) q_\pi(s, a)},$$

也可写成期望形式

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \eta, A \sim \pi} \left[\nabla_\theta \ln \pi(A \mid S, \theta) q_\pi(S, A) \right]}.$$

这里 η 是与具体指标相匹配的状态分布。

统一定理的专业读法

Theorem 9.1 是教材对多个定理的汇总。J 可以是 $\bar{v}_\pi^0$、$\bar{v}_\pi$ 或 $\bar{r}_\pi$；η 随场景变化，某些表达是严格等式，某些是教材说明的近似。因此应记住梯度的**结构**，但不能省略具体问题的指标与采样分布。

9.5 为什么梯度里会出现对数

关键恒等式是 log-derivative trick：

$$\nabla_\theta \ln \pi(a \mid s, \theta) = \frac{\nabla_\theta \pi(a \mid s, \theta)}{\pi(a \mid s, \theta)},$$

从而

$$\nabla_\theta \pi(a \mid s, \theta) = \pi(a \mid s, \theta) \nabla_\theta \ln \pi(a \mid s, \theta).$$

原来对所有动作求和的梯度因此变成 $A \sim \pi$ 下的期望，才可以用一次动作样本构造随机梯度。

softmax 的 score function

若某状态的偏好参数为 $h_1, \dots, h_K$，则对实际选择的动作 a，

$$\frac{\partial \ln \pi(a \mid s)}{\partial h_j} = \mathbf{1}\{j = a\} - \pi(j \mid s).$$

因此提高动作 a 的偏好时，归一化会同时压低其他动作的相对概率。

9.6 梯度方向的直观解释

用单样本近似真梯度可写成

$$\theta_{t+1} = \theta_t + \alpha_t \nabla_\theta \ln \pi(A_t \mid S_t, \theta_t) q_t(S_t, A_t).$$

又因为

$$\nabla_\theta \ln \pi = \frac{\nabla_\theta \pi}{\pi},$$

它可看作沿 $\pi(A_t \mid S_t, \theta)$ 的上升方向调整：估计价值为正时增强所选动作，估计价值为负时削弱所选动作。L10 会通过减去 baseline，把“绝对价值”改成“相对平均水平的优势”，从而降低方差并使解释更自然。

9.7 REINFORCE：用 Monte Carlo 回报估计动作价值

真实 $q_\pi(S_t, A_t)$ 未知。对 episodic task，使用从 t 开始的完整回报

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$$

作为样本，得到教材 Algorithm 9.1：

$$\boxed{\theta \leftarrow \theta + alpha G_t \nabla_\theta \ln \pi(A_t \mid S_t, \theta)}.$$

REINFORCE：每个 episode

1. 按当前 $\pi(\cdot \mid \cdot, \theta)$ 生成完整轨迹；
2. 对每个 t 从后向前计算回报 G_t；
3. 用 $G_t \nabla_\theta \ln \pi(A_t \mid S_t, \theta)$ 做梯度上升；
4. 新策略继续生成下一批 on-policy 轨迹。

它被称为 Monte Carlo policy gradient，因为直到 episode 结束才能获得完整 G_t，且不使用 bootstrap。

9.8 REINFORCE 的采样条件、优点与局限

策略梯度期望要求 $S \sim \eta$、$A \sim \pi(\cdot \mid S, \theta)$。教材的实用实现按当前策略生成 episode，再复用其中每个时间步；因此它是 on-policy，同时比严格独立地重新采每个状态–动作样本更节省经验。

维度	REINFORCE	L8 的 TD/Q 方法
直接更新	策略参数 θ	价值参数 w
价值样本	完整回报 G_t	一步或多步 bootstrap target
方差/偏差	通常方差较高；不由当前价值函数 bootstrap	通常方差较低，但依赖当前价值估计
更新时机	通常等 episode 结束	可逐时间步更新

容易混淆

“直接优化策略”不等于“不需要价值信息”。策略梯度定理仍用 $q_\pi(S, A)$ 给动作的概率更新加权；REINFORCE 只是用 Monte Carlo 回报估计这个价值。L10 的 actor–critic 会改用 TD critic 估计它。

9.9 初学者最容易产生的七个误解

1. **“策略梯度就是对奖励 R_t 直接求导。”**
   环境奖励与转移通常不可直接对 θ 求导；定理通过 log-policy 与 q_π 表达梯度。
2. **“η 永远等于平稳分布 d_π。”**
   不对。它由指标和折扣设定决定，也可能是折扣总访问分布。
3. **“$\ln \pi$ 是新目标函数。”**
   不是。它是把概率梯度改写为可采样期望的工具。
4. **“REINFORCE 不估计价值。”**
   它用 G_t 对 $q_\pi(S_t, A_t)$ 做 MC 估计。
5. **“softmax 输出最大的动作就是本次动作。”**
   随机策略应按输出概率采样，而不是每次都取最大者。
6. **“梯度上升一定找到全局最优策略。”**
   参数化目标通常非凸，梯度方法一般只提供局部/驻点意义的优化过程。
7. **“一条旧策略轨迹可以无修正地训练任意新策略。”**
   基础 REINFORCE 是 on-policy；使用其他行为策略的数据需要 L10 的分布修正思想。

9.10 自测题与参考答案

题目

1. 写出平均状态价值与平均奖励，并指出各自使用的状态分布。
2. 为什么 $\nabla_\theta \ln \pi$ 能把动作求和变成采样期望?
3. 写出策略梯度定理的期望形式，并说明 η 为什么不能省略。
4. 写出 REINFORCE 更新，并说明它为何属于 Monte Carlo 和 on-policy。

5. 若 $G_t < 0$，一次 REINFORCE 更新倾向于怎样改变已选动作概率?

参考答案

1. $\bar{v}_\pi = \sum_s d(s) v_\pi(s)$；$\bar{r}_\pi = \sum_s d_\pi(s) r_\pi(s)$。
2. 因为 $\nabla \pi = \pi \nabla \ln \pi$，求和中的 π 正好成为采样概率。
3. $\mathbb{E}_{S\sim\eta, A\sim\pi}[\nabla \ln \pi(A \mid S, \theta) q_\pi(S, A)]$；$\eta$ 与所优化指标及折扣情形绑定。
4. $\theta \leftarrow \theta + \alpha G_t \nabla \ln \pi(A_t \mid S_t, \theta)$；$G_t$ 是完整回报，轨迹由当前待优化策略生成。
5. 在足够小的更新下，倾向于降低该状态下已选动作的概率。

9.11 一页总结

核心结论

1. L9 从价值函数参数 w 转向策略参数 θ，用梯度上升直接改善策略。
2. 常用指标是平均状态价值和平均奖励；指标决定正确的状态加权分布。
3. 策略梯度的统一结构是 $\mathbb{E}[\nabla_\theta \ln \pi(A \mid S, \theta) q_\pi(S, A)]$。
4. 对数导数技巧把动作求和改写为可用单个动作样本估计的期望。
5. softmax 保证离散动作概率为正且和为一；动作应按概率采样。
6. REINFORCE 用完整回报 G_t 估计 q_π，是 on-policy Monte Carlo 策略梯度。
7. REINFORCE 不 bootstrap，概念简单但通常方差较高；L10 将用 baseline 和 TD critic 改进。

掌握标准：看到策略梯度公式时，能指出目标指标、状态分布、行为策略、score function 和价值估计来源。

10 L10：Actor–Critic Methods——策略与价值协同学习

L9 的策略梯度更新需要 $q_\pi(S,A)$。若用完整回报估计它，就是 REINFORCE；若改用 TD 方法持续学习价值函数，就得到 actor–critic：

- **actor** 用参数 θ 表示并更新策略，负责“怎样行动”；
- **critic** 用参数 w 表示并更新价值，负责“评价 actor 的行动”。

Actor–Critic 的最小结构

$$\text{actor:} quad\theta \leftarrow \theta + alpha_\theta \nabla_\theta \ln \pi(A \mid S,\theta) \times \text{价值信号},$$

$$\text{critic:} quad w \leftarrow w + \alpha_w(\text{TD target} - \text{当前价值})\nabla_w \hat{v} \text{ 或} \nabla_w \hat{q}.$$

它仍属于策略梯度方法，只是用 TD 函数近似实现价值估计部分。

10.1 最简单的 Q Actor–Critic（QAC）

QAC 的 critic 用带函数近似的 Sarsa 学习 $q_\pi(s,a)$。在时间 t：

$$\delta_t^Q = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, w_t) - \hat{q}(S_t, A_t, w_t).$$

actor 与 critic 分别更新为

$$\boxed{\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(A_t \mid S_t, \theta_t) \hat{q}(S_t, A_t, w_t)},$$

$$\boxed{w_{t+1} = w_t + \alpha_w \delta_t^Q \nabla_w \hat{q}(S_t, A_t, w_t)}.$$

A_t 与 A_{t+1} 都按当前策略产生，所以教材中的基础 QAC 是 on-policy。

直观理解

QAC 只把 REINFORCE 的 G_t 换成持续更新的 $\hat{q}(S_t, A_t, w)$。因此不必等 episode 结束，方差通常也比完整回报低；代价是 critic 使用 bootstrap，会受当前价值近似误差影响。

10.2 baseline invariance：为什么减去基线不改变真梯度

对任意只依赖状态、不依赖当前动作的标量基线 $b(S)$，有

$$\boxed{\mathbb{E}[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)] = \mathbb{E}[\nabla_\theta \ln \pi(A \mid S,\theta)(q_\pi(S,A) - b(S))]}.$$

原因是在固定 $S = s$ 时

$$\begin{aligned}\mathbb{E}_{A\sim\pi}[\nabla_\theta \ln \pi(A \mid s,\theta) b(s)] &= b(s) \sum_a \nabla_\theta \pi(a \mid s,\theta) \\ &= b(s) \nabla_\theta \sum_a \pi(a \mid s,\theta) = 0.\end{aligned}$$

baseline 可以依赖状态，但基础结论不允许依赖所采动作

若直接使用 $b(S,A)$，上面的动作求和一般不再为零，可能改变梯度期望。“减去任意东西都不偏”是错误说法；教材的 baseline invariance 针对 $b(S)$。

10.3 baseline 为什么有用：降低随机梯度方差

令

$$X = \nabla_\theta \ln \pi(A \mid S, \theta)[q_\pi(S, A) - b(S)].$$

改变 b 不改变 $\mathbb{E}[X]$，却会改变 $\mathrm{Var}(X)$。教材给出的逐状态最优基线为

$$b^*(s) = \frac{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta \ln \pi(A \mid s, \theta)\|^2 q_\pi(s, A)]}{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta \ln \pi(A \mid s, \theta)\|^2]}.$$

它计算复杂。去掉梯度范数权重后得到实用基线

$$b(s) = \mathbb{E}_{A\sim\pi}[q_\pi(s, A)] = v_\pi(s).$$

专业核对

baseline 的核心作用是**减方差而不改期望**，不是把梯度“变得更大”。$v_\pi(s)$ 是简洁且有清楚含义的基线，但教材明确指出它是对最优基线的实用近似，并非一般情况下的精确方差最优解。

10.4 优势函数：评价动作相对平均水平好多少

定义

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s).$$

$v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a)$ 是该状态按当前策略得到的平均动作价值，故：

- $A_\pi(s, a) > 0$：动作优于当前策略在此状态的平均水平；
- $A_\pi(s, a) < 0$：动作劣于平均水平；
- $\mathbb{E}_{A\sim\pi}[A_\pi(s, A)] = 0$。

actor 因而可用

$$\theta \leftarrow \theta + alpha_\theta \nabla_\theta \ln \pi(A \mid S, \theta) A_\pi(S, A)$$

增强正优势动作、削弱负优势动作。

10.5 用 TD error 近似优势：A2C 的关键一步

若 $v = v_\pi$，则在给定 $(S_t = s, A_t = a)$ 时

$$\mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) - v_\pi(S_t) \mid s, a] = A_\pi(s, a).$$

因此单步 TD error

$$\boxed{\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t)}$$

可作为优势的样本估计，而只需维护一个状态价值 critic。

Advantage Actor–Critic（A2C，也称 TD actor–critic）的更新是

$$\boxed{\begin{aligned} \theta_{t+1} &= \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(A_t \mid S_t, \theta_t), \\ w_{t+1} &= w_t + \alpha_w \delta_t \nabla_w \hat{v}(S_t, w_t). \end{aligned}}$$

同一个 δ_t，两种不同职责

若 $\delta_t > 0$，actor 增加已选动作的对数概率，critic 同时把 $\hat{v}(S_t, w)$ 向 TD target 上调；若 $\delta_t < 0$，actor 降低已选动作倾向，critic 向下修正当前状态价值。两次更新共享评价信号，但参数、梯度和学习目标不同。

终止状态仍约定后续价值为零：

$$\delta_t = R_{t+1} - \hat{v}(S_t, w_t) qquad (S_{t+1} \text{ 为终止状态}).$$

10.6 REINFORCE、QAC 与 A2C 的关系

方法	actor 的价值信号	critic / 估计方法
REINFORCE	G_t	无独立 TD critic；MC 回报估计 q_π
QAC	$\hat{q}(S_t, A_t, w)$	Sarsa 型动作价值 critic
REINFORCE + baseline	$G_t - \hat{v}(S_t)$	通常仍以 MC 为主
A2C	δ_t	TD 状态价值 critic；δ_t 近似 advantage

容易混淆

A2C 的 δ_t 在 critic 尚未准确时只是优势的近似样本，不等于真实 $A_\pi(S_t, A_t)$。同时，actor 改变策略后 critic 的评价目标也随之变化；这是一套耦合的随机逼近过程，不能把两个更新当成互不相关的监督学习任务。

10.7 重要性采样：用另一个分布的样本估计期望

若目标分布为 p_0、样本来自 p_1，则

$$\mathbb{E}_{X\sim p_0}[f(X)] = \mathbb{E}_{X\sim p_1}\left[\frac{p_0(X)}{p_1(X)} f(X)\right].$$

比率 $p_0(X)/p_1(X)$ 称为 importance weight。成立的覆盖条件是

$$p_0(x) > 0 \Longrightarrow p_1(x) > 0.$$

行为分布不给目标可能事件任何概率时，权重无法补回从未出现的样本。

一个离散重要性采样检查

目标分布 $p_0(+1) = p_0(-1) = 0.5$，行为分布 $p_1(+1) = 0.8, p_1(-1) = 0.2$。来自行为分布的 $+1$ 样本权重为 $0.5/0.8$，-1 样本权重为 $0.5/0.2$。稀有的 -1 样本被放大，这能修正偏差，也说明 importance weight 可能带来较大方差。

10.8 Off-policy Actor–Critic

设行为策略为 β、目标策略为 π_θ，动作重要性权重为

$$\rho_t = \frac{\pi(A_t \mid S_t, \theta_t)}{\beta(A_t \mid S_t)}.$$

教材 Theorem 10.1 在折扣情形给出

$$\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\rho, A\sim\beta}\left[\frac{\pi(A \mid S,\theta)}{\beta(A \mid S)}\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)\right],$$

其中状态分布 ρ 是从行为策略平稳分布 d_β 出发、再按目标策略转移得到的折扣总访问分布。这说明不仅动作分布需要说明，状态分布也属于定理的一部分。

用 v_π baseline 和 TD error 后，教材 Algorithm 10.3 为

$$\boxed{\begin{aligned}\delta_t &= R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t),\\ \theta_{t+1} &= \theta_t + \alpha_\theta \rho_t \delta_t \nabla_\theta \ln \pi(A_t \mid S_t, \theta_t),\\ w_{t+1} &= w_t + \alpha_w \rho_t \delta_t \nabla_w \hat{v}(S_t, w_t).\end{aligned}}$$

勘误表对应的时间下标

Chapter 10 勘误明确把 off-policy actor 更新中的策略参数统一为 θ_t：权重分子和 $\nabla \ln \pi$ 都应使用当前参数 θ_t。本文采用该修正版，并按教材 Algorithm 10.3 在 actor 与 critic 两边都乘 ρ_t。

重要性采样修正分布，但可能放大方差

当 $\beta(A_t \mid S_t)$ 很小时，ρ_t 可能很大。基础公式的无偏/一致性直觉不能自动等价为稳定、低方差的有限样本训练；同时必须保证行为策略覆盖目标策略支持集。

10.9 确定性策略梯度：策略直接输出动作

随机策略输出动作分布 $\pi(a \mid s, \theta)$；确定性策略直接输出动作

$$a = \mu(s, \theta).$$

教材 Theorem 10.2 将确定性策略梯度写成

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{S\sim\eta}\left[\nabla_\theta \mu(S,\theta) \nabla_a q_\mu(S,a)\big|_{a=\mu(S,\theta)}\right]}.$$

这是链式法则：先看 critic 认为动作往哪个方向改变会增大 q，再通过 actor 的 Jacobian 把动作方向传回策略参数。

直观理解

$\nabla_a q_\mu(s,a)$ 回答“动作稍微向哪里移动会更好”；$\nabla_\theta \mu(s,\theta)$ 回答“怎样改参数才能让输出动作向那里移动”。二者相乘就是 deterministic actor 的更新方向。

确定性梯度中没有动作随机变量 A，教材据此说明它天然适合 off-policy 学习，也适合连续动作空间。状态分布 η 仍随具体指标而定：折扣平均价值对应折扣总访问分布，不折扣平均奖励对应 d_μ。

10.10 Deterministic Actor–Critic 算法

行为策略 β 负责探索并生成 $(S_t, A_t, R_{t+1}, S_{t+1})$；目标 actor 为 μ。教材 Algorithm 10.4 使用

$$\delta_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, \mu(S_{t+1}, \theta_t), w_t) - \hat{q}(S_t, A_t, w_t),$$

$$\boxed{\begin{aligned} \theta_{t+1} &= \theta_t + \alpha_\theta \nabla_\theta \mu(S_t, \theta_t) \nabla_a \hat{q}(S_t, a, w_t)\big|_{a=\mu(S_t,\theta_t)}, \\ w_{t+1} &= w_t + \alpha_w \delta_t \nabla_w \hat{q}(S_t, A_t, w_t). \end{aligned}}$$

注意 critic 的当前项使用行为动作 A_t，而 bootstrap 项使用目标动作 $\mu(S_{t+1})$。行为策略必须有探索性；教材举例为在 μ 输出上加入噪声。使用神经网络表示 q 时，这一方向进一步形成 deep deterministic policy gradient（DDPG）的基础。

容易混淆

actor 更新中的动作必须代入 $a = \mu(S_t)$，不是直接在行为动作 A_t 处更新确定性策略；而 critic 当前预测必须评价真实执行的 (S_t, A_t)。把两处动作混成同一个会改变算法。

10.11 四类 Actor–Critic 的统一比较

方法	actor 信号	critic target	数据关系
QAC	$\hat{q}(S, A, w) \nabla \ln \pi$	$R + \gamma \hat{q}(S', A', w)$	on-policy
A2C	$\delta \nabla \ln \pi$	$R + \gamma \hat{v}(S', w)$	on-policy
Off-policy AC	$\rho \delta \nabla \ln \pi$	同上，critic 也乘 ρ	$A \sim \beta$，目标为 π
Deterministic AC	$\nabla_\theta \mu \, \nabla_a \hat{q}$	$R + \gamma \hat{q}(S', \mu(S'), w)$	可 off-policy

识别 Actor–Critic 的固定顺序

先找策略是随机的 π 还是确定性的 μ；再找 actor 使用 q、advantage/TD error，还是动作梯度；最后检查 critic 的 target、行为策略以及是否需要 importance weight。

10.12 初学者最容易产生的九个误解

1. **“Actor–Critic 是与策略梯度无关的第三类方法。”**
   它仍是策略梯度，只是价值由 TD critic 估计。
2. **“critic 决定最终执行哪个动作。”**
   actor 产生动作；critic 提供评价信号。
3. **“减 baseline 会把真梯度缩小。”**
   状态基线不改变期望，主要改变方差。
4. **“$v_\pi(s)$ 总是严格最优 baseline。”**
   教材的精确最优式含 score function 范数权重；v_π 是简洁实用选择。

5. **“TD error 就是真实 advantage。”**
   critic 准确时其条件期望对应 advantage；有限样本和近似 critic 下只是估计。

6. **“off-policy 只需把 actor 乘重要性权重。”**
   教材 Algorithm 10.3 的 critic 更新也乘同一动作权重。

7. **“行为策略可不给目标动作任何概率。”**
   这违反重要性采样覆盖条件。

8. **“确定性策略不需要探索。”**
   目标策略可确定，但收集数据的行为策略仍需探索，例如加入动作噪声。

9. **“DPG 的 actor 和 critic 都在行为动作处求梯度。”**
   critic 当前项用行为动作；actor 的动作梯度在目标动作 $\mu(S)$ 处计算。

10.13 自测题与参考答案

题目

1. REINFORCE 与 QAC 的唯一结构性差别是什么?
2. 证明状态基线 $b(s)$ 不改变策略梯度期望。
3. 写出 A2C 的 δ_t、actor 更新和 critic 更新。
4. 什么时候 $\mathbb{E}[\delta_t \mid s, a] = A_\pi(s, a)$?
5. 写出 off-policy 动作重要性权重及覆盖条件。
6. 写出确定性策略梯度的两个链式因子。
7. 在 deterministic critic target 和 actor 梯度中分别使用哪个动作?

参考答案

1. REINFORCE 用 MC 回报估计 q_π；QAC 用 TD/Sarsa critic 估计 q_π。
2. 对动作求期望后得到 $b(s)\nabla_\theta \sum_a \pi(a \mid s, \theta) = 0$。
3. $\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t)$；actor 加 $\alpha_\theta \delta_t \nabla \ln \pi$，critic 加 $\alpha_w \delta_t \nabla_w \hat{v}$。
4. 当价值函数等于真实 v_π，并按环境的一步转移对 R, S' 取条件期望时。
5. $\rho_t = \pi(A_t \mid S_t, \theta_t)/\beta(A_t \mid S_t)$；若目标概率非零，行为概率必须非零。
6. $\nabla_\theta \mu(S, \theta)$ 与 $\nabla_a q_\mu(S, a)|_{a=\mu(S)}$。
7. critic target 用 $\mu(S_{t+1})$，actor 动作梯度在 $a = \mu(S_t)$ 处；critic 当前项仍用真实行为动作 A_t。

10.14 一页总结

核心结论

1. Actor–critic 是“策略梯度 actor + TD 价值 critic”，仍属于策略梯度方法。
2. QAC 用 Sarsa critic 估计动作价值；A2C 用状态价值 TD error 估计 advantage。
3. 状态 baseline 不改变策略梯度期望，却可降低随机梯度方差。
4. $A_\pi = q_\pi - v_\pi$ 衡量动作相对当前策略平均水平的好坏。
5. A2C 的同一 δ_t 同时训练 actor 和 critic，但两者参数与梯度不同。
6. Off-policy AC 用 $\rho_t = \pi/\beta$ 修正动作采样，且要求行为策略覆盖目标策略。
7. 教材 Algorithm 10.3 的 actor 和 critic 更新均乘 importance weight。
8. DPG 的 actor 梯度为 $\nabla_\theta \mu \nabla_a q$，自然适合连续动作与 off-policy 数据。
9. 确定性目标策略仍需由探索性行为策略收集有覆盖的数据。

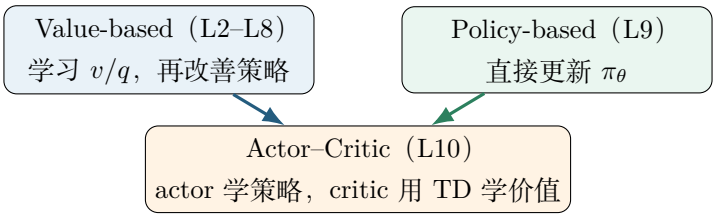
掌握标准：能够从伪代码中独立标出 actor、critic、advantage、behavior/target policy、importance weight 与动作梯度的求值位置。

L1–L10 跨章节串联

十讲各自解决什么问题

讲次	核心问题	最应记住的对象或更新	方法定位
L1	怎样描述序贯决策?	$(\mathcal{S}, \mathcal{A}, p, r, \gamma)$、$\pi$、$G_t$	MDP
L2	固定策略有多好?	$v_\pi = r_\pi + \gamma P_\pi v_\pi$	策略评价
L3	最优策略满足什么?	$v_*(s) = \max_a \mathbb{E}[R + \gamma v_*(S') \mid s, a]$	最优性
L4	模型已知怎样求解?	Bellman 算子的确定性迭代	DP
L5	模型未知怎样估价值?	完整样本回报 G_t	MC
L6	带噪方程怎样增量求根?	$x \leftarrow x - \alpha \tilde{g}(x)$	SA / SGD
L7	怎样边交互边自举?	估计 $\leftarrow$ 估计 $+\alpha$(target $-$ 估计)	TD control
L8	表格太大怎样泛化?	$w \leftarrow w + \alpha \delta \nabla_w \hat{v}/\hat{q}$	Value FA
L9	怎样直接优化策略?	$\theta \leftarrow \theta + \alpha \nabla \ln \pi\, q_\pi$	Policy gradient
L10	怎样合并策略与价值学习?	actor 更新 θ；TD critic 更新 w	Actor–Critic

三条主线最后怎样汇合



从一参数到双参数

L8 只更新价值参数：

$$w \leftarrow w + \alpha_w \delta \nabla_w \hat{v}.$$

L9 只显式更新策略参数，但需要价值样本：

$$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \ln \pi \, G.$$

L10 同时维护两者：

$$\theta \leftarrow \theta + \alpha_\theta \delta \nabla_\theta \ln \pi, \qquad w \leftarrow w + \alpha_w \delta \nabla_w \hat{v}.$$

贯穿十讲的六个判断维度

看到任何新 RL 算法，按下面顺序拆解：

1. **MDP 目标：**折扣回报、平均奖励，还是特定初始状态的价值？
2. **估计对象：**v_π, q_π, v_*, q_*，还是策略参数 θ？
3. **target/权重：**完整回报、TD target、advantage、importance weight，还是动作梯度？
4. **表示：**表格、线性特征、随机策略网络，还是确定性 actor？
5. **数据关系：**行为策略与目标策略分别是谁，状态–动作覆盖是否充分？
6. **理论边界：**结论属于 tabular/linear/nonlinear、on/off-policy、discounted/undiscounted 中哪一种？

最重要的算法谱系

从哪里开始	改变哪一部分	得到什么
完整 Bellman 期望	用完整轨迹样本替代期望	Monte Carlo
MC 完整回报	提前一步 bootstrap	TD
TD 表格	用共享参数与梯度更新	TD function approximation
Sarsa target	把实际下一动作换成 max	Q-learning
Q-learning + 神经网络	加目标网络与经验回放	DQN
策略梯度	用 MC 回报估 q_π	REINFORCE
REINFORCE	用 TD critic 估 q_π	QAC
QAC	减 v_π baseline，以 TD error 估 advantage	A2C
A2C	行为策略改为 β 并加 π/β	Off-policy AC
随机 actor	直接输出动作并用 $\nabla_a q$	Deterministic AC

十讲之后仍应坚持的专业边界

“无模型”不等于不使用 Bellman 方程；“策略方法”不等于不需要价值评价；baseline 不改变期望不代表任意 action-dependent 减项都安全；TD error 是 advantage 的估计而非恒等替代；importance sampling 能修正分布却可能增大方差；线性 on-policy 定理不能自动替神经网络 off-policy 控制提供收敛保证。

总自测

1. 从固定策略 Bellman 方程推到 TD 更新，并说明随机逼近模板在哪里。
2. 写出 MC、Sarsa、Q-learning、DQN 的 target，指出各自的自举和数据关系。
3. 写出策略梯度定理的期望结构，解释 $\ln \pi$ 与状态分布 η 的作用。
4. 从 REINFORCE 依次改出 QAC 和 A2C，说明每一步替换了什么价值信号。
5. 证明状态 baseline 不改变梯度期望，并说明 v_π 为什么有用但不总是精确最优 baseline。
6. 写出 off-policy AC 的重要性权重与覆盖条件。
7. 写出 deterministic actor 更新，并区分行为动作、target 动作与动作梯度求值位置。

内容核对与专业性说明

本合订版的知识依据限定在工作区 `Book-Mathematical-Foundation-of-Reinforcement-Learning-main`：

- 正文：`Chapter 1` 至 `Chapter 10` 的十个分章 PDF；
- 连续更新讲义：`Lecture slides/slidesContinuouslyUpdated/` 中对应的 L1–L10 PDF；
- 视频课程讲义：`Lecture slides/slidesForMyLectureVideos/` 中对应的 L1–L10 PDF；
- 勘误：`5 - Errata for the Springer version.pdf`。

L1–L8 正文复用已核对的 `RL_L1-L8_Review.pdf` 第 4–41 页；L9 与 L10 重新依据 Chapter 9–10、新版连续更新 slides、视频 slides 和勘误表整理。原 L1–L8 文件没有被覆盖。

L9–L10 重点专业校验

1. 区分平均价值 $\bar{v}_\pi^0/\bar{v}_\pi$ 与平均奖励 $\bar{r}_\pi$ 的状态分布；
2. 将 Theorem 9.1 作为多种指标的统一结构，不把 η 错写成永远固定的 d_π；
3. 明确 REINFORCE 用完整回报估计 q_π，基础版本是 on-policy Monte Carlo；
4. 区分精确最优 baseline 与实用的 v_π baseline，并限制 baseline invariance 的条件；
5. 说明 TD error 在相应条件下以条件期望表示 advantage，而非逐样本恒等；
6. Off-policy AC 的 actor 与 critic 均按教材乘 π/β，并写明支持集覆盖和方差问题；
7. DPG 中 critic 当前项用行为动作，target 与 actor 梯度在确定性目标动作处求值；
8. 按勘误表采用 $P_\pi^k \to \mathbf{1}d_\pi^\top$ 的矩阵次序，以及 Algorithm 10.3 中当前参数 θ_t。